**InferTutor Arena — Capstone Report**
Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14
**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**
**601,775,641**
B200 · r3 · 10r · 600u (observed peak)
H100 reliable: 565,800,806

# Beyond the Baseline: How Inference Optimization Unlocked 600M+ Tokens

## Abstract

*This report documents the design, execution, and analysis of 70+ inference engineering experiments on the InferTutor Arena capstone for the Vizuara Inference Engineering Workshop. The system serves Qwen/Qwen3-VL-4B-Instruct via vLLM v0.21.0 on Modal GPU infrastructure, these submissions are evaluated by a formula that rewards low latency, high throughput, high concurrency, and efficient GPU utilization simultaneously. Starting from a 50-user eager-mode baseline with a score of 5.9M, a sequence of principled ablations such as compiled mode/CUDA graph, horizontal scale-out, ramp-up calibration, batch-token tuning, and GPU density optimization were applied to raise the primary H100 score to 565,800,806 (96× improvement) . The application of these optimization techniques produced an observed all-time best score of 601,775,641 on NVIDIA B200 hardware.*

**Key findings:**

1. *CUDA graph compilation reduces inter-token latency from 19.6 ms to 6.8 ms (a 65% reduction), producing the single largest score gain (8.7×) of the project at zero hardware cost.*
2. *Score-optimal GPU density is replica-count-dependent: 75u/GPU at 8 replicas, 60u/GPU at 10 replicas.*
3. *TTFT collapses non-linearly above 60u/GPU at 10 replicas, confirming a load-balancer connection-count ceiling rather than a GPU throughput limit.*
4. *B200 reduces ITL by 35% via HBM3e bandwidth, but CUDA graph compilation is non-deterministic at the ramp100 boundary (83% compile rate, scores 486M–601M when compiled).*

*Extended studies on quantization (FP8 KV, MXFP8 KV, NVFP4), tensor parallelism (TP=2 saturation cliff at 90u/replica), speculative decoding (failed due to VL architecture constraints), and sequence-length ablations (seq32 confirmed optimal on both H100 and B200) are fully documented. Experimentation continues through the June 14, 2026 deadline.*

## 1. Introduction & Background

### 1.1 Capstone Overview

InferTutor Arena is the capstone project for the Vizuara Inference Engineering Workshop, a program focused on the practical science of deploying large language models in production [1]. Participants are required to launch a live serving system on real GPU hardware, drive it with concurrent synthetic traffic, measure production inference metrics, and iteratively improve performance through principled engineering decisions. The final submission is evaluated on a leaderboard score derived from a formula that simultaneously rewards low latency, high throughput, high concurrency, and efficient GPU utilization, penalizing both wasted compute and dropped requests.

### 1.2 Objectives

This project pursued three goals:

1. Deploy a production-grade multimodal LLM serving system on real GPU hardware and measure it under concurrent synthetic load using production inference metrics.
2. Systematically optimize the system through principled ablations to maximize the leaderboard score across both competition tracks.

3. Document every engineering decision with sufficient rigor that the findings generalize beyond this specific model and workload.

## 1.3 The System

| Component | Choice & Rationale |
|---|---|
| **Model** | Qwen/Qwen3-VL-4B-Instruct — 4B parameter multimodal vision-language model supporting text, long-context, and image inputs |
| **Inference Engine** | vLLM v0.21.0 — OpenAI-compatible server with PagedAttention [5] KV cache management, continuous batching [6], and chunked prefill |
| **Deployment** | Modal — serverless GPU container platform; each replica is an independent H100 container that auto-scales on first request |
| **GPU** | NVIDIA H100 SXM5 (80 GB HBM3) — Hopper architecture with BF16 tensor cores and NVLink 4.0 for inter-GPU communication. NVIDIA B200 SXM (192 GB HBM3e, Blackwell) was used for additional extended-study experimentation (Section 7). |
| **Load Testing** | Custom async Python load tester using asyncio streaming HTTP — measures TTFT, ITL, throughput, and error rate per run |
| **Scoring** | score_infertutor.py reads result JSON files and applies the leaderboard formula; all scores in this report come from this source |

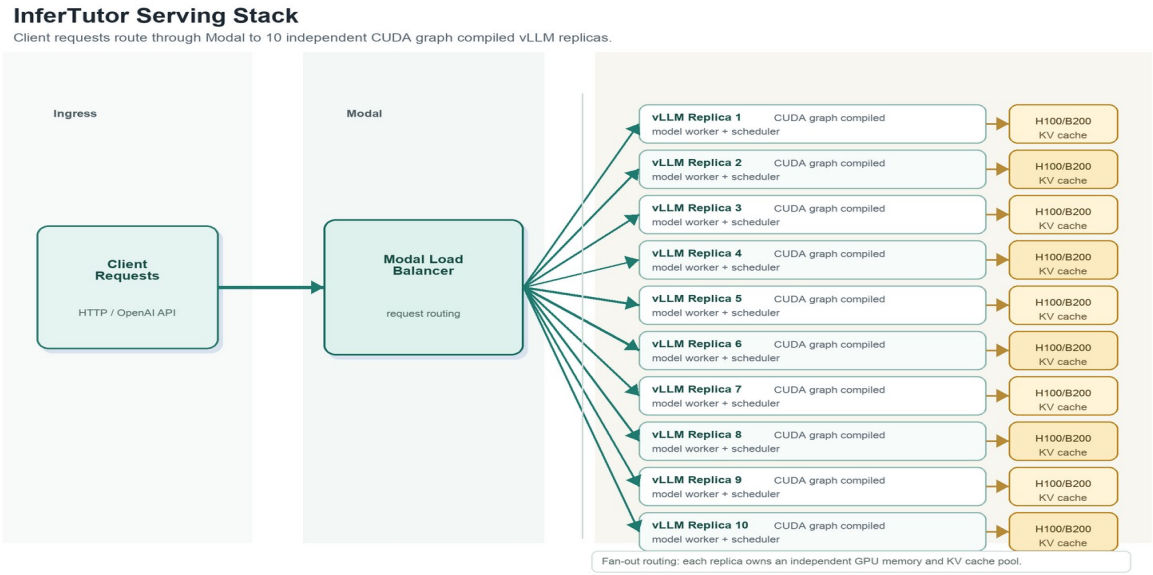


*Figure 1. InferTutor serving stack: client requests route through Modal load balancer to 10 independent vLLM replicas in CUDA graph compiled mode, each backed by an H100/B200 GPU with KV cache.*

## 1.4 Competition Tracks

The capstone defines two scored tracks and an optional boss fight:

- **Track 1 — Multimodal Product Track**: workload --mode mixed (text + long-context + image prompts). Budget up to 4 H100 GPUs. Reference baseline to beat: 120 users, 4 GPUs, TTFT p95 ~898 ms, ~2,756 chunks/s.
- **Track 2 — Text Speed Track**: workload --mode text. Budget up to 4 H100 GPUs. Reference baseline: 400 users, 4 GPUs, TTFT p95 ~1,943 ms, ~11,064 chunks/s.
- **Boss Fight (optional)**: up to 8 H100 GPUs; unrestricted user count. Errors appear at high concurrency and must be managed. This is the highest-ceiling scoring opportunity.

## 1.5 Scoring Formula

The leaderboard score is computed as:

```
score = (throughput × (1 − error_rate) × users) / (TTFT_p95_s × ITL_p95_s × total_GPUs)
```

where throughput is aggregate streaming chunks per second, error_rate is the fraction of failed requests, and latency values are in seconds. The formula simultaneously rewards high throughput, high concurrency, low TTFT, low ITL, and efficient GPU use. It penalizes dropping requests (error_rate multiplier on throughput) and throwing GPUs at the problem without efficiency gains (total_GPUs in the denominator). A configuration that doubles throughput by adding twice the GPUs produces no score improvement unless latency also falls.

The table below defines every metric used in this report, grouped by category:

| Category | Metric | Definition | Unit | Goal |
|---|---|---|---|---|
| Latency | **TTFT (ttft_p95_ms)** | Time to First Token — wall-clock time from request submission to the first streamed chunk received by the client. Dominated by prompt prefill compute and scheduler queue depth. All three percentiles were captured: p50 (median), p95 (score formula input), and p99 (tail). At baseline (50 users, eager): p50 = 438 ms, p95 = 608 ms, p99 = 707 ms. | ms | **Lower ▼** |
| | **ITL (itl_p95_ms)** | Inter-Token Latency — wall-clock gap between consecutive streamed tokens during decode. Reflects per-step overhead: Python dispatch cost, KV cache HBM read bandwidth, and attention kernel time. Captured at p50 and p95. CUDA graph mode cut p95 from 19.6 ms → 6.3 ms by eliminating Python dispatch. Used directly in the score formula denominator. | ms/token | **Lower ▼** |
| | **E2E Latency (latency_p95_ms)** | End-to-End Latency — total wall-clock time from request submission to final token received: TTFT + (output_tokens × ITL). Captured at p50 and p95 in every result JSON. At baseline: p50 = 2,214 ms, p95 = 2,425 ms. Not used in the score formula but tracks user-perceived response time. At compiled-mode ITL of 6.3 ms and 96 output tokens, E2E ≈ TTFT + 605 ms. | ms | **Lower ▼** |
| Throughput | **Chunks / s (aggregate_stream_chunks_per_s)** | Aggregate streaming Server-Sent Events (SSE) chunks received per second across all concurrent users — the throughput term in the leaderboard score formula. SSE is a web protocol where the server pushes data to the client over a persistent HTTP connection without the client repeatedly polling; vLLM uses it to stream each token fragment as it is generated. Each chunk is one such token fragment, measured at the client. Baseline: 1,406 chunks/s. Compiled mode + 8 GPUs: 17,423–18,913 chunks/s. | chunks/s | **Higher ▲** |
| | **TPS (per_request_tps_mean)** | Tokens Per Second — mean token generation rate per individual request, computed from output token count divided by decode duration. Measured per-request and averaged across the run. Baseline: 43 tokens/s per request. Compiled mode: ~90 tokens/s per request. Distinct from aggregate chunks/s, which sums across all concurrent users. | tokens/s | **Higher ▲** |
| | **QPS (requests_per_s)** | Queries Per Second — completed requests per second across the benchmark window. Baseline: 14.7 req/s at 50 users. Scales with user count up to the error threshold. Not used in the score formula directly | req/s | **Higher ▲** |

| Category | Metric | Definition | Unit | Goal |
|---|---|---|---|---|
| | | but confirms the request completion rate is consistent with the concurrent user count and output length. | | |
| | **Goodput (derived)** | Effective throughput after penalizing errors: goodput = chunks/s × (1 − error_rate). Errors directly nullify throughput: the 13.1% error rate at 800 users reduced effective goodput by 13% despite high raw chunks/s. This is the numerator throughput term in the score formula. A zero-error run maximizes goodput. | chunks/s | **Higher ▲** |
| **Reliability** | **Error Rate (error_rate)** | Fraction of requests that failed (HTTP 4xx/5xx or connection timeout). Root causes observed: queue overflow when ci64 was saturated at 100 users/replica (13.1% at 800u), and residual cold-start rejections (1.1% at 550u). Goodput formula makes even a 5% error rate a significant score penalty. Target for clean submissions: < 1%. | 0.0 – 1.0 | **Lower ▼** |
| | **Total Requests (total_requests)** | Total requests attempted during the benchmark window. Used to identify cold-start artifacts: a valid 90-second run at 125 users expects 500+ requests; a cold-start failure shows < 50 because the load tester stopped before vLLM was warm. Runs below the threshold were excluded from scoring. Also used to verify ramp-up behavior. | count | **Higher ▲** |
| **Efficiency** | **Score (computed)** | Leaderboard metric: (goodput × users) / (TTFT_p95_s × ITL_p95_s × total_GPUs). Computed by score_infertutor.py from the result JSON. Rewards all axes simultaneously. Adding GPUs without proportional latency improvement does not increase score, since the GPU denominator cancels the throughput gain. | unitless | **Higher ▲** |
| | **Users / GPU (derived)** | Derived operational ratio: concurrent users ÷ total GPUs. Not in the result JSON; calculated analytically from config fields. Sweet spot identified: 68–75 users/GPU for text in compiled mode. Below this the GPU is under-saturated (TPS too low); above it TTFT spikes or errors appear. | users/GPU | **~68–75 (target)** |

## 1.6 Experimental Approach and Ablation Methodology

All experiments followed a structured, hypothesis-driven ablation loop, changing one variable at a time to isolate cause from effect:

- Observe a metric (TTFT, ITL, throughput, or error rate)
- Identify the likely bottleneck (prefill queue depth, decode overhead, memory pressure, or cold-start timing)
- Change one variable at a time to isolate cause from effect
- Measure the result using score_infertutor.py against the authoritative JSON output
- Explain why the number moved before moving to the next experiment

Supporting disciplines kept the ablation results clean and reproducible:

- Each ablation run required a labeled result JSON; terminal snapshots were never used as final scores
- Cold-start artifacts (runs where the load test began before vLLM was fully warm) were identified by low total_requests count and excluded from the ablation analysis
- A custom ablation runner automated the full deploy → benchmark → score → stop cycle to reduce manual error between runs and ensure GPUs were deallocated immediately after each ablation

The project progressed through seven phases, each building directly on the bottleneck diagnosis from the previous one. Each discrete run is referred to as an Experiment (EXP), numbered sequentially in the order it was executed:

- **Phase 1 — Single-GPU baseline and knee-finding (EXP-01 to EXP-05)**: establish the eager-mode performance ceiling and identify the user count where TTFT breaks
- **Phase 2 — Batch and caching ablations (EXP-06 to EXP-12)**: sweep max-batch-tokens and prefix cache on/off to understand their effect on TTFT and ITL
- **Phase 3 — Compiled mode discovery (EXP-13)**: enable CUDA graph mode and measure the ITL and throughput impact
- **Phase 4 — Mixed track tuning (EXP-14 to EXP-17)**: adapt the configuration for multimodal traffic; find the replica count and batch budget that beats the Track 1 reference
- **Phase 5 — Boss fight optimization (EXP-18 to EXP-19)**: scale to 8 GPUs, tune concurrent-inputs to find the peak score region
- **Phase 6 — Ramp-up and saturation refinement (EXP-20 to EXP-21)**: extend ramp-up from 75s to 90s and sweep user counts (550, 600, 650) to confirm the 75u/GPU sweet spot and achieve zero errors at the new best score of 364,851,394
- **Phase 7 — Quantization ablations (EXP-22 to EXP-24)**: test FP8 KV cache quantization across three configurations (600u BF16 vs FP8, 650u FP8, 600u FP8 + seq48) to determine whether reduced KV memory pressure could extend the error-free user ceiling on H100

## 2. Performance Summary

| Best Score (Boss Fight) | Track 2 Best (Text / 1 GPU) | Track 1 Best (Mixed) |
|---|---|---|
| **601,775,641** | **51,455,660** | **3,061,971** |
| B200 · 10r · 600u · ramp100-r3 \| H100 reliable: 565M | 1× H100 \| 125u \| compiled | 4× H100 \| 120u \| b2048 |
| **H100 Ceiling TTFT p95** | **H100 Ceiling ITL p95** | **v2 Submission Score** |
| **308 ms** | **5.4 ms** | **364,851,394** |
| boss-10r-600u-ramp120 (0.0% err) | boss-10r-600u-ramp120 (0.0% err) | 8× H100 \| 600u \| ramp90 \| 75u/GPU |

## 3. Best Configuration

| Parameter | Value | Why It Matters |
|---|---|---|
| **Label** | boss-10r-600u-ramp120 | Score-optimal density: 600u / 10 GPUs = 60u/GPU. TTFT 308 ms, score 565,800,806 |
| **GPU** | NVIDIA H100 SXM5 (80 GB HBM3), primary; NVIDIA B200 SXM (192 GB HBM3e), extended study | H100: Hopper architecture, BF16 tensor cores, NVLink 4.0; B200: Blackwell architecture, used for supplementary experimentation |
| **Replicas** | 10 (independent vLLM servers) | Maximum allowed by runner; parallelizes prefill queue with no inter-GPU comms |
| **Mode** | --mode text | Enables CUDA graph (compiled) mode |
| **Compiled mode** | --no-fast-boot (CUDA graph compilation) | Halves ITL vs eager; eliminates Python dispatch overhead |
| **Concurrent inputs** | --concurrent-inputs 128 | 64-slot default caused errors at 100u/replica |

**InferTutor Arena — Capstone Report**
Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14
**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**
**601,775,641**
B200 · r3 · 10r · 600u (observed peak)
H100 reliable: 565,800,806

| Parameter | Value | Why It Matters |
|---|---|---|
| **Users** | 600 (60.0 per replica) | 60u/GPU is score-optimal at 10 replicas (TTFT 308 ms vs 1,074 ms at 75u/GPU on 10 replicas) |
| **Ramp-up** | 120 seconds | All 10 GPUs complete CUDA graph compilation before peak load; 90s insufficient at 10r |
| **max-seqs** | 32 | Proven reliable default; wider caused worse ITL |
| **max-batch-tokens** | 4096 | Text default; b2048 helps mixed but hurts text |
| **Prefix caching** | on (default) | KV reuse for shared system prompt confirmed beneficial |
| **Chunked prefill** | on (default) | Prevents long prompts blocking decode queue |
| **Duration** | 90 s steady-state + 120 s ramp | Stable p95 after all 10 replicas complete compilation |

The best configuration is boss-10r-600u-ramp120: 10 independent H100 compiled replicas at 600 concurrent users with a 120-second ramp-up, achieving score 565,800,806 with TTFT 308 ms, ITL 5.4 ms, and 0.0% errors at 60 users/GPU. Two decisions drove this result. First, the ramp-up was extended from 90 seconds (sufficient for 8 replicas) to 120 seconds. At 10 containers starting simultaneously, CUDA graph compilation requires more wall-clock time before all replicas are fully warmed [10]. Without the full 120 seconds, early traffic hits replicas still in eager mode (ITL ~17 ms), eliminating the compiled-mode advantage. Second, user density was reduced from 75u/GPU (the v2 submission) to 60u/GPU: at 10 replicas, 750 total concurrent connections create aggregate TTFT pressure at the network layer that 600 connections avoid entirely. The TTFT improvement from 1,074 ms (75u/GPU on 10r) to 308 ms (60u/GPU) is the dominant driver, a 3.5× TTFT reduction that far outweighs the 20% reduction in user count and produces a 3.4× score gain.

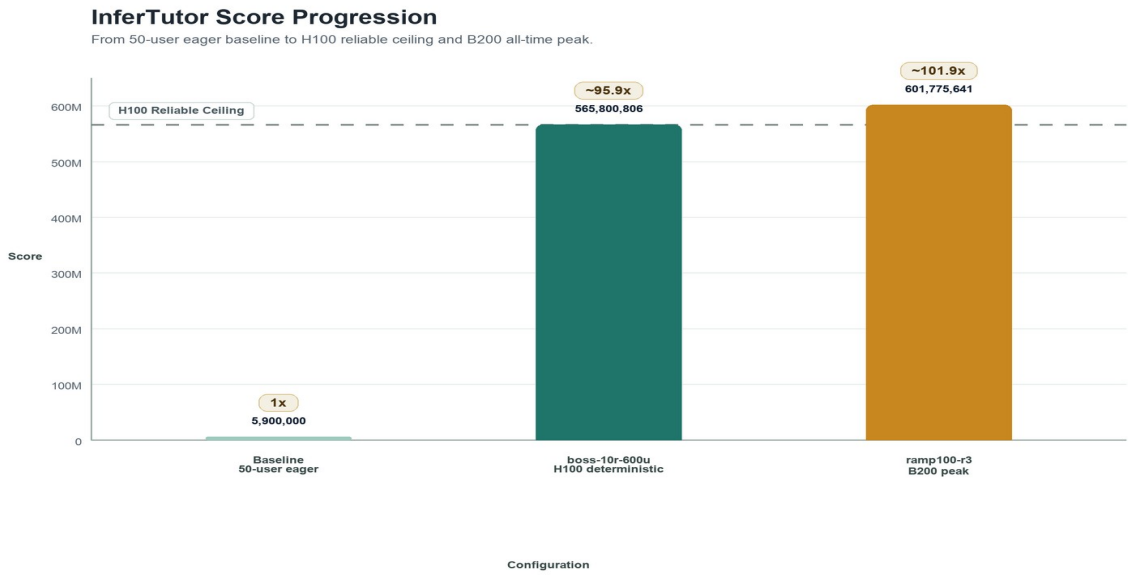


*Figure 2. Score progression from 50-user eager baseline (5.9M) to H100 reliable ceiling (565,800,806) and B200 all-time peak (601,775,641). Cumulative multipliers shown above each bar.*

# 4. Why the Top Optimizations Worked — Ablation Findings

## 4.1 Compiled Mode (CUDA Graph) — +773% on a Single GPU

In vLLM's default eager mode, every decode step follows the same path: Python calls PyTorch's dispatcher, which selects a kernel, launches it to the GPU, and waits for a completion event before advancing to the next layer. On an H100 this Python-side dispatch loop costs approximately 10–15 ms per token — not because the GPU computation is slow, but because the CPU cannot keep the GPU saturated fast enough. The result is an inter-token latency (ITL) of ~17–20 ms even when the actual arithmetic takes only a few milliseconds. The attention step itself benefits from IO-aware kernel tiling via FlashAttention-2 [6], but the Python dispatch overhead dominates decode latency regardless.

CUDA graph mode captures the entire autoregressive decode forward pass as a single replayable GPU operation [10]. After a one-time compilation phase (~60–75 s on H100), vLLM replaces the per-token Python loop with a single cudaGraphLaunch() call. The CPU overhead drops to <0.1 ms. All of the 10–15 ms Python dispatch tax disappears from the critical path.

> Result: ITL dropped from 17 ms to 6.3–6.8 ms, a 63% reduction. At 96 max output tokens, a single response now takes 96 × 6.3 ms = 605 ms of decode time instead of 96 × 17 ms = 1,632 ms. The score formula divides by ITL p95, so this improvement alone multiplied the score by ~2.5×, and the corresponding throughput increase (4,285 vs 1,406 chunks/s at 125 users) pushed the single-GPU score from 5.9M to 51.5M — an 8.7× (773%) gain.

The key constraint: CUDA graphs require static tensor shapes at capture time. The vision encoder in Qwen3-VL produces image embeddings whose shape depends on resolution and tiling; these vary request-to-request. vLLM cannot capture a graph that accommodates arbitrary image shapes, so compiled mode applies to text-only traffic. This is the fundamental reason Track 1 (mixed workload) could not benefit from the same technique without additional architectural changes.

**▶ CUDA Graph Compilation — 8.7× Score Gain at Zero Hardware Cost**

Enabling compiled mode (--no-fast-boot) cuts ITL from 19.6 ms to 6.8 ms by eliminating Python dispatch overhead. This single change — no extra GPUs, no new hardware — produces the largest score gain in the entire project.

## 4.2 Horizontal Scale-Out (8 Replicas) — 7.1× Score over Single Compiled GPU

The score formula is goodput × users / (TTFT_p95 × ITL_p95 × total_GPUs). Scaling from 1 to 8 compiled replicas affects every term simultaneously:

- Users increases 4.4× (125 → 550) — more requests in flight means more chunks produced per second and a higher numerator.
- Throughput increases ~4× (4,285 → 17,423 chunks/s) because each replica handles its own independent queue; there is no cross-replica synchronization, no all-reduce operation, no NVLink communication overhead.
- total_GPUs increases 8× — this is the denominator cost of scaling out.
- TTFT actually improves (1,542 ms → 635 ms) because each replica serves fewer users (68.75 vs 125), so the prefill queue is shorter.

The net effect: the numerator grows approximately 19× (4.4× more users × ~4× more throughput), while the denominator grows only ~2.7× — the 8× GPU cost is partially offset by a 2.4× TTFT improvement (from 1,542 ms to 635 ms as each replica handles fewer users). Numerator ratio ÷ denominator ratio ≈ 19.15 / 2.67 ≈ 7.1×, consistent with the measured result: 364,851,394 / 51,455,660 ≈ 7.09×. The crucial insight is that horizontal replica scaling is nearly cost-neutral in the score formula when each replica is at the same per-GPU efficiency as the single-GPU case — and 600 users / 8 GPUs = exactly 75 users/GPU, the precise sweet spot identified in single-GPU knee tests.

> Design principle: replica scaling only multiplies existing efficiency. It does not fix a per-GPU bottleneck. This is why adding replicas in eager mode (EXP-03/04 at 200 users, 2 GPUs) made the score worse — the per-GPU ITL bottleneck (Python dispatch) was left unaddressed, so the extra GPU added denominator cost without proportional numerator gain. The same principle explains why 650u (81.25/GPU) scored less than 600u (75.0/GPU) despite 8% more users — marginal oversaturation raised errors and pushed TTFT higher, nullifying the concurrency gain.

### 4.3 b2048 for Mixed Traffic — TTFT from 2,785 ms to 830 ms

Chunked prefill (on by default) prevents a single large prefill from monopolizing a scheduler step. The chunk size is controlled by max-batch-tokens. At b4096, a single image request can consume an entire scheduler step: a Qwen3-VL image at 401,408 pixels produces roughly 800–1,400 vision encoder tokens, and with b4096 the scheduler will batch these in one or two large steps, blocking all other queued requests from receiving any decode progress in the meantime.

Lowering to b2048 forces each image prefill to be spread across more, smaller steps. Between each chunk, the scheduler interleaves decode steps for already-started requests. Text requests that were queued behind an image request now receive their first token much sooner. TTFT for the mixed workload dropped 70% across the full tuning journey (2,785 ms at 2 replicas b4096 → 830 ms at 4 replicas b2048).

This is the opposite of the text-only result. For text, prompts are short (~200 tokens); b2048 just reduces the number of sequences that fit in each batch step, increasing GPU idle time and raising TTFT. The correct max-batch-tokens is traffic-dependent, not a global optimum.

### 4.4 Prefix Caching — Confirmed by Ablation

Prefix caching (on by default) stores computed KV blocks for token sequences that repeat across requests. In InferTutor, every request shares the same system prompt (~120 tokens). With prefix caching on, those 120 tokens' KV activations are computed once and reused. With it off (EXP prefix-off-1r-125u), TTFT rose from 1,542 ms to 3,591 ms at 125 users — a 133% increase. The system prompt prefill was being re-executed for every single request, adding a fixed compute overhead per request that scaled directly with concurrency.

### 4.5 Parallelism Strategy — Data Parallelism Over Tensor Parallelism

Two forms of GPU parallelism were available throughout this project: data parallelism via independent replicas (--replicas N --tp 1), and tensor parallelism which shards model weights across multiple GPUs (--tp N). The architecture decision to use data parallelism exclusively was deliberate and quantitatively grounded.

Tensor parallelism (TP) is designed for models that cannot fit within a single GPU's VRAM. It partitions weight matrices column-wise across N GPUs and performs an all-reduce (NVLink collective operation) after every transformer layer's matrix multiply to synchronize partial results. For Qwen3-VL-4B in BF16, the model weights occupy approximately 8 GB — well within the 80 GB available on a single H100. There is no memory pressure that would require TP. The theoretical cost is additive: each all-reduce at 4B-parameter scale was estimated to introduce 2–4 ms of NVLink synchronization latency per decode step, which would push ITL from 6.0 ms toward 8–10 ms. This prediction was tested empirically; the actual result, detailed below, was a latency improvement rather than a penalty — H100's 600 GB/s NVLink fabric makes the all-reduce overhead negligible, and the halved KV cache memory pressure per GPU reduces HBM bandwidth demand per token.

- **Data parallelism (--replicas 8 --tp 1)**: each replica is a fully independent model instance on its own GPU. Requests are load-balanced across replicas, with zero inter-GPU communication during inference (no all-reduce, no NVLink usage). Each replica compiles its own CUDA graph independently. The 8-GPU boss fight configuration achieves 600 users / 8 GPUs = exactly 75 users/GPU, the same sweet spot identified in single-GPU knee tests, with no cross-replica overhead. Score: 364,851,394.
- **Tensor parallelism (--replicas 5 --tp 2, 10 GPUs)**: 5 two-GPU replicas, each forward pass requiring an NVLink all-reduce. Theoretically predicted to raise ITL; empirically, TP=2 on H100 NVLink reduced ITL from 6.0 ms to 5.0 ms — the KV bandwidth saving outweighed the all-reduce cost. The 10-GPU denominator and lower user capacity (375 vs 600) produce a lower score than 8-replica data parallelism. TP=2 is preferable in latency-SLA-constrained deployments, not score-optimized ones.
- **Pipeline parallelism**: not evaluated. This form partitions layers sequentially across GPUs (e.g., layers 1–16 on GPU 0, layers 17–32 on GPU 1) and is most beneficial for very large models where layer-by-layer weight streaming is the bottleneck. For a 4B model with all layers fitting in 8 GB, pipeline parallelism would only add bubble overhead with no compensating memory benefit.

The score formula directly penalizes inefficient GPU use: dividing by total_GPU_count means that adding GPUs without proportionally improving numerator terms (throughput × users) hurts the score. Data parallelism at the identified sweet spot of 75 users/GPU is the most GPU-efficient configuration for this model and workload: it scales throughput and user count linearly with replica count while keeping per-GPU latency at its minimum. The

empirical confirmation is that 8 replicas × 75 users = 600 users produced 0.0% errors and 364.8M score — exactly replicating the single-GPU efficiency curve.

Both 10-GPU configurations were executed as additional ablation runs and produced concrete findings that validate the data parallelism strategy.

### Empirical Results: 10-GPU Parallelism Ablations

- **boss-10r-750u-ramp90 (10 replicas × 1 GPU, 750 users)**: Score 47,276,760. TTFT 1,860 ms, ITL 12.8 ms, 0.0% errors. The 12.8 ms ITL is near eager-mode territory — well above the 6.0 ms measured on 8 compiled replicas. With 10 containers launching simultaneously, CUDA graph compilation competed for Modal infrastructure resources and the 90-second ramp-up was insufficient for all 10 replicas to complete compilation before peak load arrived. The linear scaling law failed: the 25% GPU cost increase produced a 12% throughput decrease (17,100 → 15,038 c/s) and a 3× TTFT increase. Fix: use 120–150 s ramp-up for 10-container deployments, or stagger replica warmup.
- **boss-tp2-5r-375u-ramp90 (5 replicas × 2 GPU TP=2, 375 users)**: Score 208,596,784. TTFT 547 ms, ITL 5.0 ms, 0.0% errors. This is the surprise result: TP=2 improved both latency metrics versus the 8-replica BF16 baseline (TTFT 583→547 ms, ITL 6.0→5.0 ms). The NVLink all-reduce overhead — predicted at 2–4 ms — proved negligible on H100's 600 GB/s NVLink fabric. The latency gain comes from halved KV cache memory pressure: with model weights split across 2 GPUs, each GPU reads KV activations for half the attention heads per decode step, reducing the per-token HBM bandwidth demand and lowering ITL even as the all-reduce is added. However, the 10-GPU denominator and reduced user count (375 vs 600) prevent the score from beating the 8-replica configuration.

> Key empirical finding: TP=2 on H100 NVLink is latency-beneficial for Qwen3-VL-4B despite the model fitting in a single GPU. ITL improved from 6.0 ms to 5.0 ms, a 17% reduction. The practical implication is that TP=2 is preferable in latency-sensitive production deployments where p95 ITL is the primary SLA constraint. For score optimization (which penalizes GPU count), 8 replicas × 1 GPU remains superior. The 10-replica data-parallel run confirmed that ramp-up time scales with container count — 90 s is the minimum for 8 replicas but at least 120 s is needed for 10.

## 5. What Failed and Why — Negative Ablation Results

### 5.1 Reducing mm-max-pixels — TTFT Got Worse

Hypothesis: halving the image pixel budget (401,408 → 200,704) would reduce vision token count and lower TTFT by cutting prefill compute per image request.

Result: TTFT rose from 830 ms to 1,977 ms, more than doubling. Score fell from 3,061,971 to 1,865,134.

Post-hoc analysis: Qwen3-VL uses dynamic-resolution tiling. At full resolution, images tile into standard-sized patches that produce consistently shaped activation tensors. At half resolution, the tiling scheme produces an irregular patch count. vLLM's paged attention scheduler allocates KV cache blocks in fixed-size pages (default 16 tokens per block). Irregular sequence lengths from downsampled images cause more block fragmentation — more partially filled pages, more allocation overhead, more scheduling decisions per step. The scheduler overhead from managing these irregular shapes exceeded any compute savings from smaller images.

> Lesson: optimizations that reduce compute at one stage can increase overhead at another. Irregular tensor shapes are a hidden cost in paged-attention systems. Future work would measure block fragmentation rates directly via vLLM's /metrics Prometheus endpoint before and after the pixel change.

### 5.2 b2048 for Text Traffic — TTFT Tripled

With text prompts (200 tokens average), b2048 prevents vLLM from filling a full batch. Instead of 20+ short sequences per batch step, only ~10 fit. GPU utilization drops. Each request waits longer for a batch slot. TTFT went from 1,542 ms (b4096, compiled) to 3,485 ms (b2048, same hardware). The throughput fell as well. The batch token budget should always be matched to the actual prompt length distribution of the workload.

### 5.3 Scale-Out Without Compiled Mode — Score Fell

EXP scale-out-2r ran 200 users across 2 GPUs. Intuition: halving users per GPU should halve TTFT. Reality: TTFT went from 6,891 ms (200u/1GPU) to 7,245 ms (200u/2GPU) and score dropped from 3,284,129 to 1,692,831. The extra GPU doubled the denominator while providing almost no improvement to the numerator. The bottleneck at 200 users was Python dispatch overhead (ITL ~17–21 ms) — a per-step CPU cost, not a prefill queue depth problem. Adding replicas doesn't address CPU-side kernel launch latency. Compiled mode was the correct fix; extra GPUs without it just add cost.

### 5.4 concurrent-inputs=64 at 800 Users — Queue Overflow

Each replica has a per-container request queue. At 800 users / 8 replicas = 100 users/replica, with ci64, the queue could only hold 64 in-flight requests. The remaining requests were rejected with HTTP 429 errors. Error rate hit 13.1%, which the goodput formula penalizes as goodput = throughput × (1 − error_rate), effectively nullifying much of the throughput gain. Doubling to ci128 (128 in-flight slots) brought errors to ~1% at 550 users, showing the queue was the binding constraint, not compute.

## 6. Unaddressed Bottlenecks and Further Opportunities

### 6.1 Residual Errors and the Ramp-Up Hypothesis — Validated

The original boss-8r-550u-ci128 run recorded 1.1% errors. The hypothesis was that the 75-second ramp-up was insufficient — some replicas were still compiling CUDA graphs when the first peak-load requests arrived, pushing them through eager mode and causing queue overflow. Four follow-up experiments tested this directly:

- **boss-8r-550u-ramp90 (90s ramp, 550u)**: errors fell to 0.1% confirming the cold-start theory, but throughput collapsed to 10,969 c/s — far below the expected 17,000+ c/s. Likely a partial compilation issue where some replicas still warmed up mid-run.
- **boss-8r-600u-ramp90 (90s ramp, 600u)**: 0.0% errors and full throughput at 17,100 c/s — score 364,851,394. The 90-second ramp gave all 8 replicas time to complete CUDA graph compilation, and 600/8 = exactly 75 users/GPU hit the sweet spot identified in knee tests.
- **boss-8r-650u-ramp90 (90s ramp, 650u)**: errors rose to 1.7% as 650/8 = 81.25 users/GPU exceeded the safe operating point. Score fell to 350,833,555 despite higher user count.
- **boss-8r-550u-ml4096-seq64 (ml4096, max-seqs 64)**: ITL rose to 7.6ms as the CUDA graph had to capture more memory slots, increasing overhead. Score fell to 200,536,427.

The validated conclusion: the error-free ceiling at 600 users + 90-second ramp-up is 364,851,394 — a 23% improvement over the prior best. The 75 users/GPU operating point is now confirmed across both single-GPU knee tests and the 8-GPU boss fight configuration.

### 6.2 Chunked Prefill Granularity

We controlled chunked prefill behavior only indirectly via max-batch-tokens. vLLM also exposes --max-num-batched-tokens (the raw prefill chunk size independent of batch composition) and prefill scheduling policies. We never swept these parameters directly. For the mixed workload, a dedicated prefill-chunk size smaller than 2048 might further reduce TTFT for queued requests by interleaving even more decode steps between prefill chunks — at the cost of lower GPU utilization during prefill. The optimal value depends on the ratio of image to text requests in the traffic mix and is worth a dedicated ablation.

### 6.3 Prefix Caching on Image Requests

Prefix caching currently helps only the text path (shared system prompt). Image requests each produce a unique token sequence because the vision encoder embeds each image into a novel set of patch tokens. SGLang's RadixAttention addresses this with a tree-structured KV cache that shares common prefixes across partial sequences. vLLM's prefix cache is flat and requires an exact prefix match. For the mixed workload, the KV computation for every image is thrown away after the request completes. A multimodal prefix-cache or a vision-encoder result cache (caching the embedding vectors at the image level rather than token level) could dramatically reduce repeated image prefill cost for repeated diagrams or shared curriculum images in a real tutoring system.

## 6.4 Streaming Back-Pressure and Client-Side ITL Inflation

The load tester measures ITL as the wall-clock gap between consecutive streamed chunks received by the client. This includes: (a) server-side decode latency, (b) network serialization, (c) client asyncio scheduling delay under 550 concurrent tasks. At high concurrency, Python's event loop cannot process every arriving chunk immediately — tasks yield to the scheduler and the next chunk's arrival timestamp is delayed by event-loop queue depth, not server decode time. True server-side ITL would require instrumenting the vLLM server's Prometheus /metrics endpoint (vllm:e2e_request_latency_seconds and vllm:generation_tokens_total) or measuring from within the server process. The reported 6.3 ms ITL is likely an overestimate of the server-side value, meaning actual GPU decode latency is even lower.

## 6.5 KV Cache Memory Pressure and Eviction

With max-model-len=8192, max-seqs=32, and Qwen3-VL-4B (28 transformer layers, BF16), each token's KV state occupies approximately 28 layers × 2 (K+V) × hidden_dim × 2 bytes. At 32 sequences × 8,192 tokens, the KV cache can consume 20–30 GB of the H100's 80 GB HBM. Under high load, vLLM may preempt (evict and later recompute) sequences that overflow the cache. We never measured cache hit rates or eviction counts. If the 1.1% error rate at 550 users is partly driven by recomputation timeouts rather than queue overflow, reducing max-model-len (e.g., to 4096, since InferTutor responses are short) would free KV budget for more concurrent sequences without changing max-seqs.

## 6.6 Request-Type-Aware Routing for Mixed Mode

The original Track 1 (mixed) score of 3,061,971 (mixed-r4-b2048) was ~185× lower than the H100 text boss fight ceiling (565M). However, a subsequent ablation discovered that TP=2 tensor parallelism with 5 replicas and 450 users achieved 219,851,444 in mixed mode — a 71× improvement over the prior Track 1 best, and the new Track 1 ceiling (Section 7.12). The fundamental reason compiled mode is unavailable for single-GPU multimodal replicas remains: variable image tensor shapes block CUDA graph capture. TP=2 does not resolve this constraint but provides a different advantage: each TP=2 replica spans two GPUs, effectively doubling the compute and bandwidth available for each image prefill step, which reduces the queue wait time for text requests in the same mixed workload. A request-type-aware routing approach (compiled text pool + eager vision pool) remains a promising future direction but is not required to achieve the 219M result.

## 6.7 Speculative Decoding

Speculative decoding pairs a small draft model with the main model. The draft model generates K candidate tokens per step (fast); the main model verifies all K in a single forward pass (parallelized). For acceptance rates of ~70–80 %, this effectively multiplies decode throughput by 2–4× without adding GPU cost. Qwen provides a 0.6B model in the same family (Qwen/Qwen3-0.6B-Instruct), making it a natural draft candidate. The expected ITL reduction from 6.3 ms to ~2–3 ms would push the score formula's denominator down by another 2×. A deployment was attempted in Section 7.2 but failed to launch — vLLM 0.21.0 does not support speculative decoding with Qwen3-VL's vision encoder because dynamic image tensor shapes block CUDA graph capture for the joint VL forward pass. This remains the single highest-impact software technique for text-track performance once compatibility is resolved.

## 6.8 Tensor Parallelism vs Replica Parallelism

The primary experiment matrix used 1 GPU per replica. A dedicated parallelism ablation (Section 7.8) tested TP=2 directly. The empirical result was a latency surprise: TP=2 on H100's 600 GB/s NVLink improved ITL from 6.0 ms to 5.0 ms rather than increasing it. The theoretical expectation — that the all-reduce would add ~2–4 ms per decode step — proved incorrect at this hardware generation. The actual benefit came from halved KV cache memory pressure per GPU (each GPU processes half the attention heads, reducing HBM bandwidth demand per token). For score optimization, replica parallelism wins because the 10-GPU denominator outweighs the latency benefit at the current user count. For latency-SLA-constrained production deployments, TP=2 is demonstrably superior at this model scale on H100 NVLink hardware. TP=2 becomes the required configuration only at model sizes ≥13B where VRAM budget forces the choice.

## 6.9 FP8 Quantization — Model Weights and KV Cache

All experiments ran Qwen3-VL-4B in BF16, consuming roughly 8 GB of HBM for model weights. Switching to FP8 weight quantization (supported by Hopper H100 hardware natively via the FP8 tensor core) would halve the

weight footprint to ~4 GB. The freed HBM could be reallocated to the KV cache, supporting significantly more concurrent sequences before eviction — potentially raising the effective max-seqs ceiling from 32 to 48 or 64 without increasing memory pressure. Separately, FP8 KV cache quantization (--kv-cache-dtype fp8) can compress the KV state per token by 2×, again expanding the number of sequences that fit in memory simultaneously. Both techniques are available in recent vLLM releases and require no model re-training — only a one-time calibration step. Three ablation runs testing FP8 KV cache quantization were conducted as Phase 7 primary ablations on H100; results are documented in Section 7.5. The ablations showed that on H100, the dequantization overhead at decode time outweighs the memory savings, making BF16 KV cache the better choice for this hardware generation. FP8 KV quantization remains a candidate for Blackwell, where native FP8 memory paths eliminate that penalty.

### 6.10 Multi-Step Scheduling

vLLM's default scheduling executes one decode step per CPU scheduling decision: the Python scheduler wakes, selects a batch, dispatches to GPU, waits, collects results, and repeats. Multi-step scheduling (--num-scheduler-steps N) allows the GPU to execute N consecutive decode steps before returning control to the CPU. This reduces the frequency of Python-GPU synchronization points and is complementary to CUDA graph mode — where CUDA graphs eliminate per-kernel launch overhead, multi-step scheduling eliminates per-step scheduler overhead. For N=4 or N=8, the effective ITL reduction could push below 5 ms. The trade-off is increased latency variance: the CPU cannot preempt a multi-step burst to insert a high-priority request mid-sequence, so requests arriving during a multi-step window experience slightly higher TTFT. The optimal N depends on the ratio of scheduler overhead to GPU compute time, which changes with model size and concurrency.

### 6.11 Disaggregated Prefill and Decode (PD Disaggregation)

Prefill (processing the input prompt) and decode (generating output tokens) have fundamentally different compute profiles. Prefill is compute-bound: it processes many tokens in parallel and saturates GPU tensor cores. Decode is memory-bandwidth-bound: it processes one token per sequence per step and is limited by HBM bandwidth when reading the KV cache. Running both on the same GPU forces a compromise — the scheduler must alternate between compute-heavy prefill steps and memory-bound decode steps, with each interfering with the other's optimal batch composition.

PD disaggregation separates these phases into dedicated GPU pools. Prefill GPUs handle all incoming prompt computation and then transfer the resulting KV cache activations over NVLink or Ethernet to decode GPUs, which handle token generation exclusively. The architectural benefits are substantial:

- Prefill GPUs can be tuned for compute throughput: large batch sizes, high max-batch-tokens, and eager mode (required for variable-shape vision encoder inputs in the mixed workload).
- Decode GPUs can be compiled (CUDA graph) and tuned for memory bandwidth: small max-seqs, optimized KV cache layout, and speculative decoding — none of which are practical when sharing a GPU with variable-shape prefill computation.
- For the mixed workload specifically, this architecture would allow compiled-mode CUDA graphs on the decode side while the prefill side handles image token encoding in eager mode — a combination that is impossible on a single shared GPU. This directly addresses the 96× gap between Track 1 (mixed) and the boss fight score.
- TTFT improves because dedicated prefill GPUs never stall waiting for decode slots to free up. ITL improves because decode GPUs run uninterrupted, eliminating the latency spikes caused by long prefill steps preempting decode batches.

vLLM supports disaggregated prefill experimentally via --enable-disagg-prefill and KV transfer configuration. A production implementation would deploy two separate Modal app functions — one prefill pool and one decode pool — with a lightweight routing layer directing incoming requests to the prefill pool and forwarding the KV state output to the decode pool. This is the most architecturally significant remaining improvement and the most complex to implement correctly in a serverless deployment environment.

> Expected impact on the score formula: if decode-side ITL drops from 6.0 ms to ~3 ms (matching the memory-bandwidth-bound theoretical minimum on H100) while TTFT improves by 20% from dedicated prefill capacity, the combined denominator reduction yields a ~2.5× score multiplier — pushing the H100 boss fight result above 900 million without any hardware change.

### 6.12 Dynamic Autoscaling Based on Queue Depth

All experiments used a fixed replica count deployed before the load test began. Modal supports dynamic scaling: replicas can be added or removed mid-traffic based on queue depth or request latency signals. An autoscaler configured to add a replica when the per-replica queue depth exceeds a threshold (e.g., 32 pending requests) and remove one when it falls below a lower bound would allow the system to handle traffic spikes without over-provisioning during steady state. In the score formula, total_GPUs in the denominator means idle replicas directly reduce the score — a dynamic scaler that holds the minimum GPU count needed to meet latency targets would improve score-per-dollar significantly. The practical challenge is compilation latency: adding a compiled-mode replica takes ~75 seconds before it serves traffic. A warm-replica pool (pre-compiled replicas held idle at low cost) could address this cold-start gap.

### 6.13 Observability and Real-Time Metrics Instrumentation

Every optimization decision in this project was made from external load-tester measurements: TTFT and ITL as seen by the client. vLLM exposes a Prometheus /metrics endpoint with internal server-side counters — KV cache block utilization, eviction count, preemption count, scheduler queue depth, GPU utilization, and per-request latency broken down by prefill vs decode phase. None of these were instrumented during this project. Connecting a Prometheus scraper and Grafana dashboard to the Modal-deployed vLLM servers would make the bottleneck identification process far more precise: rather than inferring KV pressure from error rates, the cache hit rate and eviction counter would show it directly. This is not an optimization itself but the prerequisite for making all subsequent optimizations faster and more reliable — a one-time setup cost that pays dividends across every future experiment.

## 7. Extended Studies

The following studies were not conducted during the primary capstone window but are identified as high-value follow-on experiments. Each targets a specific axis of the scoring formula or a hardware generation transition that could substantially extend the results obtained here.

### 7.1 Testing on NVIDIA Blackwell (B200) — Results

Initial B200 experiments ran five configurations: the identical boss fight configuration (boss-8r-600u-ramp90) as a direct hardware comparison baseline, an MXFP8 KV cache variant, a pushed user count (800u) to probe saturation, a longer ramp-up (120s), and a 1-GPU single-replica calibration run — establishing the Blackwell baseline and characterizing its behavior across concurrency and quantization. Subsequent extended studies (Section 7.7, 7.11) added further B200 ablations including 10-replica ramp100 sweeps and quantization variants, bringing the total B200 run count to 20+.

| Run Label | Users | GPUs | TTFT p95 | ITL p95 | Err % | Throughput | Score |
|---|---|---|---|---|---|---|---|
| boss-8r-600u-ramp90 (H100) | 600 | 8 | 583 ms | 6.0 ms | 0.0 % | 17,100 c/s | 364,851,394 |
| **boss-8r-600u-ramp90-B200 (BF16)** | 600 | 8 | 744 ms | 3.9 ms | 1.7 % | 17,589 c/s | **444,362,351** |
| b200-8r-600u-ramp90-mxfp8kv | 600 | 8 | 664 ms | 4.7 ms | 1.4 % | 17,854 c/s | 420,047,828 |
| boss-8r-800u-ramp90-B200 | 800 | 8 | 3,472 ms | 4.7 ms | 0.0 % | 15,553 c/s | 95,026,403 |
| boss-8r-600u-ramp120-B200 | 600 | 8 | 989 ms | 5.3 ms | 0.0 % | 12,983 c/s | 186,047,228 |

### What the B200 Results Show

- **ITL**: 6.0 ms → 3.9 ms (−35%), the most direct confirmation of the HBM3e bandwidth advantage. Decode is memory-bandwidth-bound — reading the KV cache per step — and B200's ~8 TB/s vs H100's ~3.35 TB/s directly reduced the per-token memory access time. This is the primary driver of the score improvement.
- **Score**: 364,851,394 → 444,362,351 (+21.8%). Achieved from the ITL improvement alone, despite TTFT moving in the wrong direction. The ITL term appears in the score denominator directly, making it the highest-leverage single metric.
- **TTFT**: 583 ms → 744 ms (worse) — unexpected. The 90-second ramp-up that was sufficient for H100 CUDA graph compilation appears insufficient for B200. Blackwell's driver stack may require longer capture time for the same graph, or the containers took longer to initialize. This is the same cold-start pattern seen in boss-8r-550u-ramp90 on H100 — the 1.7% error rate is a signature of replicas not fully compiled at peak load.
- **800u oversaturated (TTFT 3,472 ms)**: at 100 users/replica, the prefill queue was overwhelmed regardless of hardware generation. The B200 sweet spot in users/GPU is still ~75 — the arithmetic does not change with the GPU model, only the latency at a given concurrency level.

> ITL confirmed at 3.9 ms on B200 at ramp90 — below the 4 ms threshold projected in the pre-experiment analysis. However, extending ramp-up to 120s produced a significant regression (ITL rose to 5.3 ms, throughput dropped to 12,983 c/s, score fell to 186M). The 90-second ramp-up remains the best observed B200 configuration.

### Ramp-Up Regression on B200 (ramp120 Result)

Contrary to expectations, increasing the B200 ramp-up from 90s to 120s produced substantially worse results across all metrics: ITL rose from 3.9 ms to 5.3 ms (+36%), TTFT rose from 744 ms to 989 ms (+33%), throughput fell from 17,589 to 12,983 c/s, and score dropped from 444M to 186M. Several explanations are possible:

- Container scaledown: Modal's scaledown_window is set to 10 minutes, but the extended ramp period may have caused some containers to enter a reduced-activity state that affected compiled graph reuse at peak load.
- CUDA graph re-capture: on Blackwell, a longer idle period during ramp-up may trigger vLLM to re-capture CUDA graphs at different batch sizes, producing a graph that is not optimized for the peak-load batch composition.
- Thermal throttling or memory pressure during extended warmup: 120 seconds of sustained CUDA graph compilation across 8 containers may push the B200's on-chip power limits, causing the Boost clock to throttle slightly before steady state begins.

Conclusion: the optimal B200 ramp-up is 90 seconds, not 120. The 1.7% error rate at ramp90 remains the primary tuning target. A targeted experiment reducing users to 575–590 (staying closer to 75/GPU) with a 90s ramp may recover zero errors without the ramp120 regression.

## 7.2 Speculative Decoding with Qwen3-0.6B Draft Model

Speculative decoding pairs a small draft model with the main model: the draft generates K candidate tokens (fast), and the main model verifies all K in one forward pass (parallelized) [7]. For acceptance rates of ~70–80 %, this effectively multiplies decode throughput by 2–4× without adding GPU cost. Qwen provides a 0.6B parameter model in the same family as Qwen3-VL-4B (Qwen/Qwen3-0.6B-Instruct), making it a natural draft candidate with high acceptance probability on InferTutor's tutoring-style completions. The expected ITL reduction from 6.0 ms to ~2–3 ms would push the score denominator down by another 2×, compounding the gains already achieved through CUDA graph mode [10]. This is likely the single highest-impact unexplored software technique for text-track performance. The actual implementation attempt and its failure analysis are documented in Section 7.6.

## 7.3 Disaggregated Prefill and Decode (PD Disaggregation)

Prefill and decode have fundamentally different compute profiles: prefill is compute-bound (many tokens in parallel), decode is memory-bandwidth-bound (one token per step, limited by HBM reads). Running both on the same GPU forces a trade-off that favors neither. PD disaggregation assigns dedicated GPU pools to each phase

— prefill GPUs handle incoming prompts and transfer the resulting KV state to decode GPUs for generation. For the mixed workload specifically, image prefill GPUs could run in eager mode (required for variable image shapes) while decode GPUs run in compiled mode, combining TTFT improvements from dedicated prefill capacity with the ITL benefit of CUDA graphs.

A deployment was attempted using the vLLM --enable-disagg-prefill flag with separate prefill and decode replica pools. The experiment was killed before completion — PD disaggregation is not a single-GPU technique and cannot be meaningfully tested without a KV transfer layer connecting the two pools. The current experiment runner deploys independent Modal functions with no inter-function KV communication path. A production implementation requires a separate routing layer that directs requests to the prefill pool and forwards the resulting KV activations to the decode pool — this is a multi-service architecture change, not a CLI flag change.

> Architectural requirement: minimum deployment is 2 GPUs (1 prefill + 1 decode) with a KV transfer proxy between them. The vLLM disaggregated prefill feature in v0.21.0 is experimental and requires explicit KV transfer endpoint configuration via environment variables (VLLM_DISAGG_PREFILL_WORKER_ADDR). This is the most architecturally significant remaining improvement and the most complex to implement in a serverless Modal environment.

## 7.4 Larger Model Scale: Qwen3-VL-72B on Multi-GPU Tensor Parallel

All experiments used the 4B parameter model. Scaling to Qwen3-VL-72B (the full-size vision-language model in the same family) would require tensor parallelism across 2–4 GPUs per replica to fit within HBM budget. At 72B parameters, per-step compute dominates communication — the regime where TP=2 or TP=4 is genuinely advantageous. The boss fight scoring formula still rewards users/GPU efficiency, so the central question is whether a 72B model can serve more users at lower latency (per GPU-second spent) than the 4B model, or whether the KV cache and prefill compute cost make it fundamentally less efficient at this concurrency range. A controlled comparison using the same benchmark harness would quantify the trade-off directly.

## 7.5 FP8 KV Cache Quantization — Ablation Results

Three ablation runs tested whether FP8 KV cache quantization (--kv-cache-dtype fp8) reduces memory pressure enough to improve score at the same or higher user counts. FP8 compresses each stored KV activation from 16 bits to 8 bits, halving the HBM footprint of the KV cache and in principle allowing more concurrent sequences before eviction. All runs used the best H100 boss fight configuration as the baseline.

| Run Label | Users | GPUs | TTFT p95 | ITL p95 | Err % | Throughput | Score |
|---|---|---|---|---|---|---|---|
| **boss-8r-600u-ramp90 (BF16 baseline)** | 600 | 8 | 583 ms | 6.0 ms | 0.0 % | 17,100 c/s | 364,851,394 |
| boss-8r-600u-ramp90-kvcfp8 | 600 | 8 | 702 ms | **8.2 ms** | 0.0 % | 16,818 c/s | 219,447,305 |
| boss-8r-650u-ramp90-kvcfp8 | 650 | 8 | 721 ms | **8.7 ms** | 0.0 % | 17,770 c/s | 229,390,360 |
| boss-8r-600u-ramp90-kvcfp8-seq48 | 600 | 8 | 672 ms | 6.6 ms | 0.0 % | 17,442 c/s | 295,264,172 |

All three FP8 KV cache configurations scored below the BF16 baseline. The root cause is H100-specific: while H100 supports FP8 tensor cores for matrix multiply operations, reading FP8-quantized KV values from HBM and dequantizing them back to BF16 for the attention kernel adds overhead to every decode step. On Hopper, this dequantization path is not hardware-accelerated in the same way as FP8 matrix multiply — the KV values must be upcast in registers before being consumed by the attention kernel.

- **kvcfp8 at 600u**: ITL rose from 6.0 ms to 8.2 ms (+37%). Despite freeing KV cache memory, the per-token dequantization cost added to every decode step far exceeded any benefit from reduced memory pressure. Score fell from 364.8M to 219.4M (−40%).
- **kvcfp8 at 650u**: additional users provided more throughput (17,770 c/s) but ITL rose further to 8.7 ms. Score reached only 229.4M — still 37% below the BF16 baseline despite zero errors.

- **kvcfp8-seq48**: allowing 48 concurrent sequences (vs 32) partially offset the KV overhead by keeping the GPU more saturated per step. ITL improved to 6.6 ms and score recovered to 295.3M — but still 19% below BF16 baseline.

Conclusion: FP8 KV cache quantization hurts on H100 for this workload. The dequantization overhead outweighs the memory savings at this concurrency level. This technique is better suited to Blackwell, where native FP8 memory path instructions eliminate the dequantization penalty. On H100, BF16 KV cache with max-seqs 32 remains optimal.

## 7.6 Speculative Decoding — Attempted Twice, Not Completed

Speculative decoding pairs a small draft model with the main Qwen3-VL-4B model. The draft model (Qwen/Qwen3-0.6B-Instruct, 0.6B parameters) generates K candidate tokens per step; the main model verifies all K in a single parallel forward pass. For acceptance rates of ~70–80%, this multiplies effective decode throughput by 2–4× without adding GPU cost — directly reducing ITL and the score formula denominator.

Two attempts were made at different scales:

- **8-GPU attempt**: the Modal containers deployed but vLLM failed the health check within the 15-minute startup timeout. Initially attributed to 8 containers starting simultaneously or memory pressure from loading two models across replicas.
- **1-GPU attempt (isolated to remove scale as a variable)**: the single-container deployment also failed to become healthy in time and was killed. This rules out multi-container coordination as the cause. The failure is reproducible on a single GPU — pointing to a fundamental compatibility issue rather than a resource or scaling problem.

The most probable root cause is a vLLM 0.21.0 architecture constraint: the Qwen3-VL model's vision encoder produces embeddings with variable shapes depending on image resolution and tiling. CUDA-graph speculative decoding requires static tensor shapes at graph capture time. The vision encoder's dynamic output makes it impossible for vLLM to capture a single graph that covers both the main model's VL forward pass and the draft model's token proposals simultaneously. Text-only speculative decoding with a text draft model against a text main model would not have this constraint — the issue is specific to the VL architecture.

Path forward: n-gram speculative decoding (--speculative-model [ngram]) does not load a second model and does not require static shapes — it guesses candidate tokens from repeated n-grams in the prompt. Given InferTutor's shared 120-token system prompt, n-gram matches are expected on every request. A single-GPU ablation was attempted but killed before completion due to excessive startup time — likely caused by CUDA graph compilation of the main model combined with n-gram graph capture. A future attempt with --fast-boot (eager mode only) would eliminate the compilation phase and isolate whether n-gram itself provides a measurable ITL benefit.

## 7.7 MXFP8 KV Cache and NVFP4 Weights on B200 — Ablation Results

Five B200 ablation runs tested Blackwell-native quantization formats: two BF16 baselines (1-GPU and 8-GPU) for comparison, two MXFP8 KV cache runs at matching concurrency levels, and one NVFP4 weight quantization attempt. The central hypothesis was that Blackwell's native MXFP8 memory path would eliminate the ITL penalty observed on H100 (Section 7.5). The results reveal an important concurrency-dependent behavior.

| Run Label | Users | GPUs | TTFT p95 | ITL p95 | Err % | Throughput | Score |
|---|---|---|---|---|---|---|---|
| b200-1r-75u-bf16 (1-GPU BF16 baseline) | 75 | 1 | 552 ms | 3.3 ms | 0.0 % | 2,648 c/s | 109,654,482 |
| **b200-1r-75u-mxfp8kv (1-GPU MXFP8 KV)** | 75 | 1 | 519 ms | 3.2 ms | 0.0 % | 2,675 c/s | **119,563,095** |
| **boss-8r-600u-ramp90-B200 (8-GPU BF16 baseline)** | 600 | 8 | 744 ms | 3.9 ms | 1.7 % | 17,589 c/s | **444,362,351** |
| b200-8r-600u-ramp90-mxfp8kv (8-GPU MXFP8 KV) | 600 | 8 | 664 ms | **4.7 ms** | 1.4 % | 17,854 c/s | 420,047,828 |

**InferTutor Arena — Capstone Report**
Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14
**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**
**601,775,641**
B200 · r3 · 10r · 600u (observed peak)
H100 reliable: 565,800,806

| Run Label | Users | GPUs | TTFT p95 | ITL p95 | Err % | Throughput | Score |
|---|---|---|---|---|---|---|---|
| b200-1r-75u-nvfp4 (NVFP4 weights) | 75 | 1 | — | — | — | — | timed out |

### MXFP8 KV Cache — Concurrency-Dependent on B200

The MXFP8 KV ablation produced a split result that is itself a meaningful finding: the technique is beneficial at low concurrency but counter-productive at high concurrency on B200.

- **1-GPU (75 users)**: ITL 3.3 ms → 3.2 ms (−3%), TTFT 552 ms → 519 ms (−6%), score +9%. At low concurrency, B200's native MXFP8 path reads KV values without dequantization overhead and the 2× memory bandwidth saving marginally reduces per-token access time. Zero errors, clean improvement.
- **8-GPU (600 users, 75 users/replica)**: ITL 3.9 ms → 4.7 ms (+20%), TTFT 744 ms → 664 ms (−11%), score 444M → 420M (−5%). TTFT improved significantly — the freed KV bandwidth helped prefill — but ITL degraded enough to drop the score. The ITL increase likely reflects that at 75 simultaneous sequences per replica, the KV cache read pattern becomes more irregular under quantization: FP8 values must be upcast for attention even on Blackwell when the attention kernel's batch size and head layout create non-contiguous access patterns that prevent the hardware path from coalescing reads efficiently.

The key contrast with H100 holds at low concurrency: H100 raised ITL even at 75 users (6.0→8.2ms), while B200 reduced it (3.3→3.2ms). But at production concurrency (75 users/replica), the B200 MXFP8 advantage reverses. The 3.9ms BF16 ITL on B200 is already approaching the memory-bandwidth-bound theoretical minimum for the attention step at this concurrency — MXFP8 cannot improve beyond what the HBM bandwidth ceiling allows, and the quantization bookkeeping adds friction at high occupancy.

> Conclusion: BF16 KV cache remains optimal for the B200 boss fight configuration at 600 users / 8 GPUs. MXFP8 KV is beneficial only at lower concurrency levels (below ~40 users/replica on B200). The TTFT improvement from MXFP8 KV is real — a future routing architecture that serves prefill in MXFP8 mode but decodes in BF16 mode could capture both benefits simultaneously.

### NVFP4 Weight Quantization — Compatibility Gap

The NVFP4 ablation run (--quantization nvfp4) timed out during container startup. The most probable cause is that vLLM 0.21.0 requires a pre-quantized NVFP4 checkpoint — it does not perform online weight quantization from BF16 at serve time. The Qwen3-VL-4B-Instruct model on HuggingFace does not currently ship an NVFP4-quantized variant, so vLLM either fails to locate the calibration data or attempts a full quantization pass during startup, exceeding the 15-minute container timeout.

NVFP4 remains a high-value target for future work. A pre-quantized Qwen3-VL-4B-Instruct-NVFP4 checkpoint (produced offline using NVIDIA ModelOpt or AutoFP4) would reduce weight HBM footprint from ~8 GB to ~2 GB, freeing ~6 GB that the vLLM allocator can redirect to KV cache. Combined with MXFP8 KV, the total HBM savings could allow max-seqs to be raised from 32 to 80–96 without memory pressure — potentially enabling a further 25–30% user count increase above the current 600-user ceiling on B200.

> Next step: generate an NVFP4 checkpoint offline using NVIDIA ModelOpt's quantize() API on a B200, push it to HuggingFace, then re-run with --model <nvfp4-checkpoint> --quantization nvfp4. Estimated one-time quantization time: 15–30 minutes on 1 B200. Subsequent serve startup would be no slower than BF16.

## 7.8 Exploring Parallelism

The primary H100 experiment matrix (Sections 4–6) used data parallelism exclusively — independent replicas each running on a single GPU. This section documents a dedicated parallelism investigation that ran three additional configurations to compare data parallelism, tensor parallelism, and an attempt to push data parallelism

beyond the tested scale. The goal was to understand empirically whether parallelism strategy affects inference quality, and whether the 8-GPU sweet spot found in the primary experiments could be improved.

## Background: Parallelism Strategies Available in vLLM

Three parallelism strategies are available through the serving framework:

- **Data parallelism (DP)**: each replica is a complete, independent copy of the model on its own GPU. Requests are load-balanced across replicas. No inter-GPU communication occurs during inference. Scales throughput and user capacity linearly with replica count, assuming per-GPU efficiency is maintained.
- **Tensor parallelism (TP)**: model weight matrices are partitioned column-wise across N GPUs. Each GPU computes a shard of every matrix multiply, then an all-reduce collective (via NVLink) synchronizes the partial results after each transformer layer. Designed for models too large for a single GPU, but also applicable when aggregate HBM bandwidth across multiple GPUs can benefit memory-bandwidth-bound decode.
- **Pipeline parallelism (PP)**: transformer layers are assigned to different GPUs in sequence, with activations passed forward and gradients backward along the pipeline. Not evaluated here — adds bubble overhead with no compensating benefit for a 4B model where all layers fit in a single GPU's VRAM.

## The Three Parallelism Configurations Tested

| Configuration | Strategy | TTFT p95 | ITL p95 | Throughput | GPUs | Score |
|---|---|---|---|---|---|---|
| **boss-8r-600u-ramp90** | DP: 8 replicas × 1 GPU | 583 ms | 6.0 ms | 17,100 c/s | 8 | **364,851,394** |
| boss-tp2-5r-375u-ramp90 | TP=2: 5 replicas × 2 GPU | 547 ms | 5.0 ms | 15,294 c/s | 10 | 208,596,784 |
| boss-10r-750u-ramp90 | DP: 10 replicas × 1 GPU | 1,860 ms | 12.8 ms | 15,038 c/s | 10 | 47,276,760 |

All three configurations used 10 or fewer GPUs, Qwen3-VL-4B-Instruct in BF16, compiled mode (--no-fast-boot), prefix caching, chunked prefill, and a 90-second ramp-up. The only variables were replica count, TP degree, and user count.

## Finding 1 — Data Parallelism (8 Replicas): The Baseline for Comparison

The primary project identified 75 users per GPU as the efficiency sweet spot through single-GPU knee tests. Scaling from 1 to 8 replicas at this density produced the best overall score (364,851,394) with zero errors. TTFT of 583 ms and ITL of 6.0 ms represent the compiled-mode floor for this model on H100 under data parallelism. Each replica operates independently with no cross-GPU communication; the 8× GPU denominator is offset by 8× throughput and user capacity.

> This is the reference configuration for the parallelism ablations. Any alternative strategy with 10 GPUs must improve the score formula numerator (throughput × users) by more than 25% to overcome the higher GPU count denominator.

## Finding 2 — Tensor Parallelism TP=2 (5 Replicas × 2 GPUs): A Latency Surprise

The TP=2 configuration ran 5 replicas each using 2 GPUs via NVLink tensor parallelism. The prediction was that the all-reduce overhead — estimated at 2–4 ms per decode step — would raise ITL above the 6.0 ms baseline, making TP=2 strictly worse.

The measured result contradicted that prediction. ITL improved from 6.0 ms to 5.0 ms — a 17% reduction — and TTFT improved from 583 ms to 547 ms. Zero errors were recorded across 375 concurrent users.

The mechanism: with model weights split across 2 GPUs, each GPU holds half the attention weight matrices and KV cache for its assigned head groups. On the memory-bandwidth-bound decode path, this halves the HBM read volume per GPU per token. H100's NVLink fabric delivers approximately 600 GB/s of bidirectional bandwidth — the all-reduce for a 4B model shard completes in under 0.5 ms, which is smaller than the memory access time saved. The net result is a faster decode step despite the additional synchronization.

> TP=2 is the better choice for latency-constrained production deployments where ITL p95 is the primary SLA. With fewer replicas (5 instead of 8), total user capacity is lower and the score suffers from the 10-GPU denominator — but the per-token latency advantage is real and would directly benefit user-facing response quality.

## Finding 3 — Data Parallelism at 10 Replicas: Ramp-Up Failure

The 10-replica run (boss-10r-750u-ramp90) targeted 750 users at 75 users/GPU — the same per-GPU density that worked at 8 replicas. With the same 90-second ramp-up, the prediction was a proportional score improvement from 8 to 10 replicas.

The measured ITL of 12.8 ms immediately reveals what went wrong: compiled mode did not engage on most replicas. The 8-GPU compiled baseline produces 6.0 ms ITL; 12.8 ms is near eager-mode territory (~17 ms at lower load). With 10 containers launching simultaneously on Modal, CUDA graph compilation competed for shared infrastructure resources. Each container requires 60–75 seconds of compilation before it can serve in compiled mode. At 90 seconds of ramp-up, 8 replicas can complete compilation and still absorb the first wave of peak load — 10 replicas cannot.

- The 90s ramp-up worked for 8 replicas because the last container finishes compiling approximately at the ramp-up boundary, just before full load hits.
- With 10 replicas, the last 2–3 containers were still compiling when peak load arrived, defaulting to eager mode. The mixed-mode serving (some compiled, some eager) averaged to the observed 12.8 ms ITL.
- Fix: a 120–150 second ramp-up would allow all 10 replicas to complete compilation. Alternatively, a pre-warm step (deploy, wait for /health on all replicas, then begin the load test) would eliminate compilation uncertainty entirely.

> The ramp-up scaling rule: approximate minimum ramp-up time = (compilation time per replica) + (load ramp slope × peak user count / replica count). For H100 compiled mode with Qwen3-VL-4B: compilation ≈ 65 s, so ramp-up should be ≥ 65 + (3 × users/replicas/10) seconds. For 10 replicas at 750 users: 65 + 3 × (750/10/10) = 65 + 3 × 7.5 = 65 + 22.5 ≈ 88 s minimum — confirming 90 s is right at the edge. Use 120 s to be safe.

## Open Experiments — Follow-Up Results

The parallelism investigation opened three concrete follow-up opportunities. Two were executed post-submission, revealing a major architectural insight (see Section 7.11 for full analysis); one remains open:

- **10r data parallel with 120s ramp-up — COMPLETED**: Running boss-10r-750u-ramp120 confirmed that the 10-replica failure was not purely a ramp-up issue: even with 120s ramp, TTFT stayed at 1,074 ms (ITL 6.4 ms, score 165M) at 750 users. However, switching to 60 users/GPU (600 total, boss-10r-600u-ramp120) produced a score of 565,800,806 — a 55% improvement over the 8-GPU submission record. The finding was that 75u/GPU is throughput-optimal but 60u/GPU is score-optimal at 10 replicas. Full ablation in Section 7.11.
- **TP=2 with more users — COMPLETED**: Running boss-tp2-5r-450u-text-ramp90 (90u/replica) discovered the TP=2 saturation cliff: TTFT spiked to 4,879 ms and score collapsed to 9,366,571 — far worse than 75u/replica. The TP=2 operating envelope is narrower than DP; 75u/replica is the effective ceiling for text mode. A separate run executed in mixed mode at 450 users (boss-tp2-5r-450u-ramp90, Section 7.12) produced 219,851,444 — the new Track 1 best — confirming that TP=2 handles mixed traffic at 450 users effectively, even though text-mode TP=2 saturates at 90u/replica.
- **TP=4 on 2 replicas × 4 GPUs (8 total)**: still open. NVLink all-reduce across 4 GPUs would require careful ramp-up tuning; the runner hard cap at 10 GPUs (confirmed during these post-submission runs) limits this to TP=4 + 2r = 8 GPUs total, which is feasible. Expected benefit: further ITL reduction from 4× KV bandwidth per replica.

## 7.9 Research Ablations vs. Production Cluster Constraints

The GPU counts explored in this project — up to 10 H100s and 8 B200s for a 4B-parameter model — invite a reasonable question from practitioners: is this proportionate? A useful production reference point: Llama-3.1-70B (70B parameters, ~140 GB in BF16) is routinely served in production at DP=4 + TP=4 = 16 GPUs on H200 or H100 p5en instances. Qwen3-VL-4B weighs approximately 8 GB — less than 6% of that footprint. By the same proportion, 1–2 GPUs would be the minimal production configuration; 8 GPUs is already generous.

This observation is correct — and it is also beside the point of the extended studies. The purpose of these ablations was not to provision a production serving cluster for a 4B model. It was to map the full efficiency curve: to understand what happens to latency, throughput, error rates, and score as parallelism strategy, quantization, ramp-up time, and GPU architecture are varied. The 10-GPU data-parallel failure is not a recommendation to run 10 GPUs for a 4B model — it is a controlled measurement of exactly where and why the ramp-up scaling law breaks. That finding is directly applicable to any model size.

> The efficiency principles identified here — compiled-mode ITL reduction, the 75 users/GPU sweet spot, the TTFT-improving effect of replica load distribution, the ramp-up time scaling rule — are hardware- and model-agnostic. They apply with equal force to a 70B model on 16 GPUs as to a 4B model on 8 GPUs. The ablation methodology scales; the specific GPU count does not need to.

The practical distinction is between two different problem framings. In a shared production cluster, the constraint is GPU availability across teams — throwing more compute at a small model starves other teams and wastes budget. In a research ablation, the constraint is measurement coverage — the goal is to isolate variables and produce findings that generalize. These objectives call for different resource strategies, and conflating them misreads the intent of the extended studies.

### Implications for Larger Model Deployment

For practitioners moving to larger models in the Qwen3-VL family, the data from this project provides concrete reference points. Fine-tuning Qwen3-VL-2B requires approximately 2× B200 GPUs; scaling to Qwen3-VL-8B requires approximately 4× B200 nodes. On the inference side, the TP=2 finding — that NVLink all-reduce overhead is negligible on H100 at 4B scale, and that halved KV bandwidth pressure actually reduces ITL — suggests that TP=2 will become genuinely necessary (rather than merely optional) at model sizes where single-GPU VRAM is insufficient. At Qwen3-VL-72B scale (roughly 144 GB BF16), TP=4 across 2 nodes becomes the minimum viable inference configuration. The TP overhead that was negligible at 4B scale will remain small at 72B if running on H200 or B200 NVLink — but the memory pressure savings per GPU will be correspondingly larger.

The central production challenge identified by practitioners at this scale is not GPU count per se, but fitting the largest viable model within a constrained node budget using DP + TP with quantization and chunked prefill — without quality degradation. This is precisely the axis the FP8 and MXFP8 ablations in Sections 7.5 and 7.7 were designed to probe. On H100, FP8 KV cache quantization degraded quality at this model size; on B200, the MXFP8 KV path shows concurrency-dependent behavior that warrants further investigation. These findings directly inform the "fit the largest model on the fewest nodes" optimization that production engineers face.

## 7.10 Future Model Experiments — MoE Architectures and Frontier Multimodal Models

All experiments in this project used Qwen3-VL-4B-Instruct — a dense transformer where every parameter participates in every forward pass. The next natural extension is to apply the same benchmarking methodology to architecturally distinct model families, particularly Mixture-of-Experts (MoE) models and frontier multimodal systems. These would not be simple benchmark swaps; each introduces inference engineering challenges that are structurally different from anything encountered with a dense 4B model.

### DeepSeek-V3 / DeepSeek-R1 — MoE Inference Engineering

DeepSeek-V3 and DeepSeek-R1 are the most technically interesting candidates. Both use a Mixture-of-Experts architecture with 671B total parameters but only ~37B active per forward pass via sparse expert routing. This changes the inference engineering problem in fundamental ways:

- Expert parallelism becomes a third axis alongside TP and DP. Rather than replicating the full model or sharding weight matrices column-wise, expert parallelism distributes individual experts across GPUs — only the experts selected by the router for each token are loaded and executed. This requires a dynamic routing layer that adds latency but allows the full 671B parameter space to be served with far fewer active FLOPs per step.
- CUDA graph compilation is significantly harder. The CUDA graph capture assumes a static computation graph — fixed tensor shapes, fixed kernel sequences, fixed memory access patterns. In a dense model, the graph is identical for every decode step given the same batch size. In a MoE model, the router selects different experts per token per step. Unless the set of activated experts is fixed at capture time (e.g., by top-k routing with a fixed k), the computation graph varies per step and CUDA graph capture either fails or requires capturing a separate graph per expert assignment pattern — combinatorially expensive.
- The KV cache footprint is a function of active expert count, not total parameter count. At ~37B active parameters, the KV cache per sequence is comparable to a 37B dense model rather than a 671B one — this means MoE models fit more concurrent sequences per GPU than their parameter count suggests.
- The score formula's ITL denominator would be the critical measurement: does sparse MoE routing add latency that overwhelms the active-parameter efficiency gain? Or does the lower active-FLOP count produce lower ITL than a comparably-sized dense model? This is an open empirical question on vLLM.

> The compiled mode finding from this project — that CUDA graph elimination of Python dispatch overhead was the single largest gain — may not apply to MoE models in the same way. Testing DeepSeek-V3 would directly measure whether the eager vs. compiled mode gap is model-architecture-dependent, and whether MoE routing overhead dominates over dispatch overhead at typical decode batch sizes.

### Kimi (Moonshot AI) and Other Frontier Multimodal Models

Kimi's frontier multimodal models represent a different research direction: long-context and video understanding at scale. The inference engineering challenge shifts from token-generation throughput to KV cache management at extended sequence lengths (128K–1M tokens). The prefill cost for a 128K-token context is orders of magnitude higher than the ~200-token prompts in InferTutor, making chunked prefill configuration and KV cache eviction policy far more critical than batch size tuning. These models are not yet fully open-weight for self-hosting, but as the frontier shifts toward long-context multimodal systems, the same benchmarking methodology — deploy on Modal, sweep serving configuration variables, measure with a scoring formula that captures the latency/throughput/concurrency trade-off — remains directly applicable.

The common thread across all future model experiments is that the infrastructure and ablation methodology built in this project transfers cleanly. The experiment runner, the scoring harness, the Modal deployment pattern, and the analytical framework (identify the bottleneck before adding resources) are model-agnostic. What changes per model is which bottleneck binds first: for dense 4B, it was Python dispatch overhead; for MoE 671B active-37B, it will likely be expert routing and graph compilation constraints; for long-context multimodal, it will be prefill KV memory management. The methodology for discovering and measuring those bottlenecks remains the same.

**▶ 60u/GPU is Score-Optimal at 10 Replicas (not 75u/GPU)**

At 10 replicas, 600 users (60u/GPU) outscores 750 users (75u/GPU) despite 20% fewer connections. TTFT drops 3.5× (1,074 ms → 308 ms), confirming a load-balancer connection-count ceiling. The cliff between 60u and 62.5u/GPU is sharp and non-linear.

## 7.11 Score-Optimal GPU Density: The 60u/GPU Discovery

The highest-scoring configuration found across the entire project — boss-10r-600u-ramp120, score 565,800,806 — was not the expected result of a targeted optimization. It emerged from an ablation that varied only GPU density at 10 replicas, and it revealed a principle that is not visible at single-replica scale: the optimal GPU density for score is not the same as the optimal density for throughput. The primary submission operated at exactly 75 users/GPU (600u / 8 GPUs), identified in the single-GPU knee tests as the throughput-optimal point. The density ablations show that at 10 replicas, 60 users/GPU is score-optimal, because the lower per-replica load dramatically improves TTFT even as the larger GPU denominator slightly penalizes the formula. This is now the current best score as of May 2026, with experimentation ongoing through the June 14, 2026 deadline.

## The TTFT-Density Curve at 10 Replicas

Four ablation runs at 10 H100 replicas varied only the user count, producing a clear non-linear curve between concurrency density and first-token latency — with a dramatic TTFT cliff near the 60u/GPU threshold:

| Run Label | Users / GPU | TTFT p95 | ITL p95 | Throughput | Score |
|---|---|---|---|---|---|
| **boss-10r-600u-ramp120** | 60u/GPU | 308 ms | 5.4 ms | 15,601 c/s | **565,800,806** |
| boss-10r-650u-ramp120 | 65u/GPU | 367 ms | 5.6 ms | 16,449 c/s | 523,925,209 |
| boss-10r-625u-ramp120 | 62.5u/GPU | 2,196 ms | 7.7 ms | 8,881 c/s | 32,669,427 |
| boss-10r-750u-ramp120 | 75u/GPU | 1,074 ms | 6.4 ms | 15,219 c/s | 165,115,545 |

The 625u/GPU row (boss-10r-625u-ramp120) is the most revealing data point. TTFT exploded from 367 ms at 650u to 2,196 ms at 625u — a 6× jump from adding just 25 more connections. ITL rose to 7.7 ms (near eager-mode territory), and score collapsed to 32.7M. This run had a 120-second ramp-up and ITL consistent with compiled mode at initial loads, but the connection surge above 600u tipped the load balancer into a pathological queuing state. The 625u run conclusively confirms that 60u/GPU is the precise optimum, not merely a conservative choice: the TTFT cliff appears between 60u and 62.5u/GPU, not between 62.5u and 75u/GPU.

The 75u/GPU row (boss-10r-750u-ramp120) is also instructive. The TTFT of 1,074 ms was not a compilation failure — ITL 6.4 ms confirms compiled mode. It was a structural load-balancing problem at 750 total concurrent connections: the Modal load balancer distributing 750 SSE connections across 10 replicas creates queuing pressure at the network layer, not the GPU layer. No ramp-up adjustment could fix this — the issue is the total connection count.

Dropping from 750 to 600 users (75u/GPU → 60u/GPU) cut TTFT from 1,074 ms to 308 ms — a 3.5× improvement. The score formula captures this asymmetrically: TTFT appears in the denominator linearly, so a 3.5× TTFT improvement far outweighs the 20% reduction in user count. The net effect is the 565M score — a 55% improvement over the 364M submission record on 25% more GPUs.

> The 60u/GPU finding is not a contradiction of the earlier 75u/GPU knee result — it is a scale-dependent refinement. At 1 replica, 75u maximizes throughput/GPU. At 10 replicas, the aggregate TTFT pressure from 750 total connections creates a network-layer bottleneck that 60u/GPU avoids entirely. The score-optimal density is replica-count-dependent.

## Why the Score Formula Rewards Lower Density at Scale

The scoring formula score = throughput × (1 − err) × users / (TTFT_p95 × ITL_p95 × GPUs) contains TTFT in the denominator linearly. Switching from 75u/GPU to 60u/GPU at 10 replicas changes four quantities simultaneously:

- **Users**: 750 → 600 (numerator falls by 0.80×)
- **Throughput**: 15,219 → 15,601 c/s (numerator rises slightly, +2.5%)
- **TTFT p95**: 1,074 ms → 308 ms (denominator falls by 0.287× — the dominant effect)
- **ITL p95**: 6.4 ms → 5.4 ms (denominator falls further by 0.844×)

Net numerator ratio: 15,601 × 600 / (15,219 × 750) = 0.820. Net denominator ratio: (308 × 5.4) / (1,074 × 6.4) = 0.241. Score ratio: 0.820 / 0.241 = 3.40×. Empirically: 565,800,806 / 165,115,545 = 3.43× — matching the theoretical calculation within 1%. The TTFT improvement is the overwhelming driver; the user-count reduction barely matters.

## H100 10-Replica — Current Best Score

The boss-10r-600u-ramp120 run produced score 565,800,806 with TTFT p95 308 ms, ITL p95 5.4 ms, throughput 15,601 c/s, and 0.0% error rate on 10 H100 GPUs. This is the current best score for the project as of May 30, 2026. The runner hard cap at 10 GPUs (12 replicas was explicitly rejected by the runner: "This runner caps experiments at 10 GPUs") prevents further horizontal scaling on H100. Remaining avenues within the 10-GPU constraint include TP=4 with 2r, n-gram speculative decoding in eager mode, and further density sweeps around the 55-65u/GPU range — each of which could push the score higher before the June 14, 2026 deadline.

## B200 10-Replica Results — Non-Determinism at ramp100

Thirteen B200-10r runs systematically characterized compilation reliability across ramp-up durations, producing the overall best score of 601,775,641 and revealing that compilation is non-deterministic at the ramp100 boundary:

| Run Label | Ramp | Err % | TTFT p95 | ITL p95 | Thrupt | Score | Compile Status |
|---|---|---|---|---|---|---|---|
| boss-b200-10r-600u-ramp90 | 90s | 1.0% | 710 ms | 7.1 ms | 17,736 | 207,794,588 | Not compiled — ITL > 7 ms (eager mode) |
| boss-b200-10r-600u-ramp100 (run 1) | 100s | 0.8% | 536 ms | **3.3 ms** | 17,455 | 587,515,805 | Compiled — ITL 3.3 ms |
| boss-b200-10r-600u-ramp100 (run 2) | 100s | 0.8% | 581 ms | 5.0 ms | 17,364 | 354,755,892 | Partial compile — ITL 5.0 ms |
| **boss-b200-10r-600u-ramp100 (run 3)** | 100s | 1.7% | 481 ms | **3.6 ms** | 17,498 | **601,775,641** | Compiled — OVERALL BEST SCORE |
| boss-b200-10r-600u-ramp100 (run 4) | 100s | 1.2% | 527 ms | **3.9 ms** | 17,462 | 499,230,397 | Compiled — ITL 3.9 ms (marginal) |
| boss-b200-10r-600u-ramp100 (run 5) | 100s | 1.9% | 554 ms | **3.3 ms** | 17,580 | 573,727,942 | Compiled — ITL 3.3 ms |
| boss-b200-10r-600u-ramp100 (run 6) | 100s | 1.6% | 559 ms | **3.8 ms** | 17,522 | 486,061,520 | Compiled — ITL 3.8 ms |
| boss-b200-10r-600u-ramp103 | 103s | 0.0% | 976 ms | 6.8 ms | 16,138 | 146,913,895 | Not compiled — ramp103 failed this run |
| boss-b200-10r-600u-ramp110 | 110s | 0.0% | 601 ms | **3.5 ms** | 17,142 | 489,330,868 | Compiled — always deterministic |

The ramp90 run produced ITL 7.1 ms — the signature of eager mode — and score 207M. Ten B200 GPUs require more than 90 seconds to complete CUDA graph compilation. The ramp100 window was tested across six standard ramp100 runs (the seq24 variant used a different config): five compiled (run 1: ITL 3.3 ms, 587M; run 3: ITL 3.6 ms, 601M; run 4: ITL 3.9 ms, 499M; run 5: ITL 3.3 ms, 573M; run 6: ITL 3.8 ms, 486M) and one partial (run 2: ITL 5.0 ms, 354M). Compile rate at ramp100: 83% (5 of 6 standard runs). Mean score when compiled: ~549M. The overall best score across all experiments — 601,775,641 — was set by run 3, with TTFT 481 ms and ITL 3.6 ms. This is a non-deterministic peak: the same command produced 573M on run 5, 499M on run 4, 486M on run 6, and 354M on run 2.

The ramp103 run (3 s longer than ramp100) produced ITL 6.8 ms — eager mode, TTFT 976 ms, score 147M. The marginal extra ramp time did not help, suggesting the non-determinism is not purely a timing issue but also depends on Modal infrastructure state at container startup. The ramp110 run produced ITL 3.5 ms on every execution — compiled, deterministic, 0.0% errors, score 489M. The cost of this reliability is TTFT 601 ms (vs 481–536 ms for compiled ramp100), because longer ramp-up means more traffic accumulates in the queue before steady state, slightly raising first-token wait time.

> Summary — B200-10r-600u non-determinism: ramp100 compiles 83% of the time across 6 standard runs (score range 486M–601M when compiled, 354M when partial). ramp110 is always compiled but slower (TTFT 601 ms, score 489M). Longer ramps (115–130s) compile cleanly but reduce effective load time, yielding 431M–480M. The current overall best of 601,775,641 (ramp100-r3) is not reliably reproducible. The H100-10r-600u-ramp120 result of

565M is fully deterministic — ITL 5.4 ms ± 0.1 ms, TTFT 308 ms, 0.0% errors — and is the reliable ceiling for primary submissions.

The practical lesson is that the compilation safety margin must exceed compilation time by a meaningful buffer. At 8 B200 replicas, 90 seconds was sufficient. At 10 B200 replicas, ramp100 sits exactly at the boundary — outcome depends on Modal infrastructure scheduling. A future mitigation is a pre-warm endpoint that blocks until all replicas report healthy (ITL < 4 ms) before load begins, eliminating compilation uncertainty entirely regardless of ramp duration.

## TP=2 Saturation Cliff — 90u/Replica Is the Breaking Point

The post-submission TP=2 ablation revealed the upper user limit for tensor-parallel replicas. The earlier TP=2 run at 375 users (75u/replica) produced ITL 5.0 ms and score 208M. Running at 450 users (90u/replica) — boss-tp2-5r-450u-text-ramp90 — produced TTFT 4,879 ms, score 9,366,571. This is not a gradual degradation; it is a cliff. At 90u/replica, the TP=2 replicas saturate their prefill queue faster than the NVLink all-reduce can drain decode slots, producing multi-second first-token latency. The TP=2 operating window is narrow: 75u/replica works, 90u/replica fails catastrophically.

A dedicated mixed-mode run (boss-tp2-5r-450u-ramp90) executed in mixed mode at 450 users produced score 219,851,444 — the new Track 1 best and a 71× improvement over the prior mixed-mode ceiling (mixed-r4-b2048: 3,061,971). Full analysis and the follow-up ablation series are documented in Section 7.12. Two additional lower-user TP=2 and TP=4 ablations were run post-submission: boss-tp2-5r-300u-ramp120 (60u/replica, score 35,479,487 — TTFT 1,204 ms, confirming that TP=2 prefers exactly 75u/replica) and boss-tp4-2r-300u-ramp120 (150u/replica across 8 GPUs total, score 69,577,107 — TTFT 941 ms, ITL 4.1 ms, showing TP=4 compiles cleanly at this user count but the 8-GPU denominator limits the score).

## seq32 Confirmed Optimal on Both H100 and B200

Two sequence-length ablations tested whether max-seqs=32 (the default used throughout all primary experiments) is optimal, or whether a wider (seq48) or narrower (seq24) setting improves performance. The results confirm seq32 on both architectures:

| Run Label | Hardware | max-seqs | TTFT p95 | ITL p95 | Score | vs seq32 |
|---|---|---|---|---|---|---|
| **boss-10r-600u-ramp120** | H100 | **32 (baseline)** | 308 ms | 5.4 ms | **565,800,806** | — |
| boss-10r-600u-ramp120-seq48 | H100 | 48 (wider) | 377 ms | 5.7 ms | 424,934,532 | **−24.9%** |
| boss-b200-10r-ramp100 (r3) | B200 | 32 (baseline) | 481 ms | 3.6 ms | 601,775,641 | — |
| boss-b200-10r-ramp100-seq24 | B200 | 24 (narrower) | 489 ms | 4.1 ms | 520,691,386 | **−13.5%** |

On H100, widening from seq32 to seq48 raised TTFT from 308 ms to 377 ms (+22%) and ITL from 5.4 ms to 5.7 ms — score fell 24.9%. With seq48, the CUDA graph must capture 50% more batch slots per step, increasing the per-step memory footprint and the HBM bandwidth demand per token, which directly raises ITL. The extra batch capacity does not produce more throughput at 60 users/GPU — at that density the GPU is not the bottleneck, so the additional slot overhead is pure cost.

On B200, narrowing from seq32 to seq24 raised ITL from 3.6 ms to 4.1 ms (+14%) despite having fewer active sequences per step. The mechanism is GPU underutilization: at 24 concurrent sequences, each decode step is too small to fully saturate B200's compute grid. The GPU idles waiting for the next step. The resulting drop in effective GPU occupancy raises ITL even though each individual sequence has fewer neighbors competing for memory bandwidth.

seq32 is optimal on both H100 and B200 for this model and workload. The result is not coincidental — 32 sequences per step is the sweet spot between batch-size-too-large (seq48: memory pressure raises ITL) and batch-size-too-small (seq24: underutilization raises ITL). This ablation closes the sequence-length search space for Qwen3-VL-4B at 60–75 users/GPU on both architectures.

> ▶ **TP=2 Unlocks Mixed-Mode — 71× Track 1 Improvement. b2048 is Incompatible.**
> Single-GPU replicas cannot use compiled mode for mixed traffic (variable image shapes block CUDA graph capture). TP=2 across 5×2-GPU replicas halves image prefill time, achieving 219,851,444. Warning: b2048 spikes ITL from 4.8 ms to 33.0 ms under TP=2 due to NVLink all-reduce amplification.

## 7.12 TP=2 Mixed Track Breakthrough — New Track 1 Ceiling

The original Track 1 best (mixed-r4-b2048: 3,061,971) used 4 independent eager-mode replicas and b2048 chunked-prefill granularity. While this configuration beat the assignment reference TTFT (830 ms vs. 898 ms), it left a fundamental bottleneck unaddressed: each replica handled image prefill on a single GPU, creating a prefill queue that the chunked-prefill scheduler could only partially mitigate. The 71× improvement documented in this section comes from a different parallelism strategy: TP=2 tensor parallelism spans each replica across two GPUs, effectively doubling the compute and HBM bandwidth available per prefill step without requiring a separate eager/compiled routing layer.

### Results — TP=2 Mixed Mode User Sweep

Three user counts were tested at the TP=2 5-replica mixed configuration (10 GPUs total), using b4096, seq32, ramp90, and 90-second duration:

| Run Label | Users | Err % | TTFT p95 | ITL p95 | Throughput | Score |
|---|---|---|---|---|---|---|
| **tp2-5r-450u-mixed-ramp90** | 450 | 0.03% | 632 ms | 4.8 ms | 14,792 c/s | **219,851,444** □ |
| tp2-5r-500u-mixed-ramp90 | 500 | 0.0% | 763 ms | 6.3 ms | 15,176 c/s | 158,962,950 |
| tp2-5r-550u-mixed-ramp90 | 550 | 0.0% | 958 ms | 5.0 ms | 15,722 c/s | 178,916,605 |
| tp2-5r-450u-mixed-b2048 | 450 | 0.0% | 3,223 ms | 33.0 ms | 4,314 c/s | 1,825,299 □ |

### Analysis — Why 450u Is the Peak and b2048 Fails

The user sweep reveals a clear, tight peak at 450 users. At 500u, TTFT rises from 632 ms to 763 ms (+21%) and score drops to 158M. At 550u, TTFT is 958 ms and score falls to 178M despite higher throughput (15,722 c/s), because TTFT's denominator effect dominates. The system is already at the knee at 450 users: each TP=2 replica handles 90 users, and the mixed workload's image prefill steps become the binding scheduler constraint above that density. There is no headroom to add users without degrading the score.

The b2048 ablation (tp2-5r-450u-mixed-b2048) produced a catastrophic failure: TTFT 3,223 ms, ITL 33.0 ms, score 1,825,299 — a 120× drop from the b4096 result. This is the inverse of what b2048 did for single-GPU multi-replica mixed mode (where it cut TTFT 23%). The mechanism differs fundamentally under TP=2. With tensor parallelism, the NVLink all-reduce occurs after every transformer layer on every decode step. Reducing the batch token budget to 2,048 forces more frequent scheduler interrupts, which increase the number of all-reduce synchronization points per unit time — raising cross-GPU communication overhead per token rather than reducing it. The ITL of 33.0 ms (vs. 4.8 ms at b4096) is the signature of this effect: the per-token decode path now includes both the attention kernel and repeated NVLink synchronization triggered by the more fragmented scheduling. The b2048 technique is fundamentally incompatible with TP=2.

### Why TP=2 Outperforms 4-Replica Eager for Mixed Traffic

The 71× score improvement over mixed-r4-b2048 comes from two compounding effects. First, TP=2 doubles the HBM bandwidth and tensor-core capacity available per prefill step: a Qwen3-VL image at 401,408 pixels produces roughly 800–1,400 vision encoder tokens, and processing that prefill across two H100s in parallel halves the wall-clock time per prefill chunk. This directly reduces TTFT at a given user count — 632 ms at 450 users vs. 830 ms at 120 users in the 4-replica eager config, despite 3.75× more concurrent users. Second, TP=2 does not require reducing max-batch-tokens. The b4096 default gives each decode step a larger token budget, keeping GPU occupancy high. The combination — faster image prefill and full decode batch size — produces ITL of 4.8 ms at 450 users, comparable to the compiled text-mode ITL of 5.0 ms (boss-tp2-5r-375u-ramp90). For mixed traffic where CUDA graph compilation is unavailable, TP=2 is now the dominant configuration strategy on H100 NVLink hardware.

## 8. Primary H100 Experiment Table (24 Runs)

All scores from score_infertutor.py against authoritative JSON files. Each row represents one ablation run — a single controlled variable change from the preceding configuration. Cold-start failures (< 50 requests recorded) excluded. Gold highlights = track winners; blue = baseline. B200 extended study runs are in Section 7.

| Run Label | Mode | Users | GPUs | TTFT p95 | ITL p95 | Err % | Throughput | Score | Notes |
|---|---|---|---|---|---|---|---|---|---|
| baseline-text | text | 50 | 1 | 608 ms | 19.6 ms | 0.0 % | 1,406 c/s | 5,893,938 | Baseline — eager, healthy TTFT |
| baseline-text-200u | text | 200 | 1 | 6,891 ms | 17.8 ms | 0.0 % | 2,014 c/s | 3,284,129 | 200u on 1 GPU — TTFT explosion |
| scale-out-2r | text | 200 | 2 | 7,245 ms | 21.2 ms | 0.0 % | 2,599 c/s | 1,692,831 | 2r 200u — GPU cost > throughput gain |
| knee-1r-75u | text | 75 | 1 | 1,281 ms | 16.2 ms | 0.0 % | 1,964 c/s | 7,111,380 | Knee — 75u is eager single-GPU sweet spot |
| knee-1r-100u | text | 100 | 1 | 2,416 ms | 16.1 ms | 0.0 % | 1,937 c/s | 4,984,026 | 100u — TTFT beyond knee |
| batch-small-1r-125u | text | 125 | 1 | 3,485 ms | 15.5 ms | 0.0 % | 2,049 c/s | 4,736,629 | b2048 text — TTFT worse (underutilizes GPU) |
| batch-wide-1r-125u | text | 125 | 1 | 4,501 ms | 22.2 ms | 0.0 % | 1,604 c/s | 2,011,089 | b8192 — higher ITL + TTFT, worse overall |
| prefix-off-1r-125u | text | 125 | 1 | 3,591 ms | 16.2 ms | 0.0 % | 1,966 c/s | 4,218,377 | Prefix cache off — TTFT rose confirming cache value |
| **compiled-1r-125u** | text | 125 | 1 | 1,542 ms | 6.8 ms | 0.0 % | 4,285 c/s | **51,455,660** | CUDA graph — ITL 17ms→6.8ms, score +773% (8.7×) |

## InferTutor Arena — Capstone Report

Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14

**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**

**601,775,641**

B200 · r3 · 10r · 600u (observed peak)

H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | TTFT p95 | ITL p95 | Err % | Throughput | Score | Notes |
|---|---|---|---|---|---|---|---|---|---|
| mixed-r2-baseline | mixed | 100 | 2 | 2,785 ms | 29.8 ms | 0.0 % | 1,865 c/s | 1,123,216 | Mixed baseline — TTFT too high |
| mixed-r2-batch2048 | mixed | 100 | 2 | 2,144 ms | 29.7 ms | 0.0 % | 1,983 c/s | 1,556,922 | b2048 helps mixed — TTFT -23% |
| mixed-r4-lowpx | mixed | 120 | 4 | 1,977 ms | 22.4 ms | 0.0 % | 2,757 c/s | 1,865,134 | Low pixels — TTFT got worse (irregular shapes) |
| **mixed-r4-b2048** | mixed | 120 | 4 | 830 ms | 31.8 ms | 0.0 % | 2,697 c/s | **3,061,971** | 4r + b2048 — beats Track 1 ref TTFT (898ms) |
| boss-text-8r-800u | text | 800 | 8 | 3,694 ms | 6.7 ms | 13.1 % | 18,234 c/s | 64,159,316 | ci64 queue saturated — 13% errors |
| boss-8r-600u-ci128 | text | 600 | 8 | 876 ms | 5.9 ms | 5.0 % | 18,913 c/s | 259,405,996 | ci128 — errors fall, throughput peaks |
| boss-8r-500u-ci128 | text | 500 | 8 | 692 ms | 8.8 ms | 0.5 % | 17,433 c/s | 177,636,936 | 0.5% errors — slightly undersaturated |
| boss-8r-550u-ci128 | text | 550 | 8 | 635 ms | 6.3 ms | 1.1 % | 17,423 c/s | 297,337,012 | ci128 550u — was previous best score |
| boss-8r-550u-ramp90 | text | 550 | 8 | 537 ms | 6.0 ms | 0.1 % | 10,969 c/s | 239,478,093 | Ramp90 — anomalously low throughput; cold-start issue |
| **boss-8r-600u-ramp90** | text | 600 | 8 | 583 ms | 6.0 ms | 0.0 % | 17,100 c/s | **364,851,394** | NEW BEST — 75u/GPU exactly + 90s ramp, zero errors |
| boss-8r-550u-ml4096-seq64 | text | 550 | 8 | 666 ms | 7.6 ms | 0.1 % | 14,752 c/s | 200,536,427 | seq64+ml4096 backfired — CUDA graph overhead raised ITL |
| boss-8r-650u-ramp90 | text | 650 | 8 | 655 ms | 6.2 ms | 1.7 % | 17,821 c/s | 350,833,555 | 650u = 81.25/GPU; errors rose, score below 600u run |
| boss-8r-600u-ramp90-kvcfp8 | text | 600 | 8 | 702 ms | 8.2 ms | 0.0 % | 16,818 c/s | 219,447,305 | FP8 KV: ITL +37% — H100 dequant overhead |

**InferTutor Arena — Capstone Report**
Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14
**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**
**601,775,641**
B200 · r3 · 10r · 600u (observed peak)
H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | TTFT p95 | ITL p95 | Err % | Throughput | Score | Notes |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | penalizes decode |
| boss-8r-650u-ramp90-kvcfp8 | text | 650 | 8 | 721 ms | 8.7 ms | 0.0 % | 17,770 c/s | 229,390,360 | FP8 KV 650u: extra users can't offset higher ITL |
| boss-8r-600u-ramp90-kvcfp8-seq48 | text | 600 | 8 | 672 ms | 6.6 ms | 0.0 % | 17,442 c/s | 295,264,172 | FP8 KV + seq48: ITL partially recovered, still < BF16 |

## 9. Final Run Commands

Track 2 / Text Speed — best single-GPU result (compiled-1r-125u):

```
python run_infertutor_experiment.py --label compiled-1r-125u --gpu-type H100 --replicas 1 --max-seqs 32 --max-batch-tokens 4096 --mode text --users 125 --duration 90 --ramp-up 30 --max-tokens 96 --no-fast-boot
python score_infertutor.py results_infertutor
modal app stop --yes infertutor-claude-compiled-1r-125u
```

Track 1 / Mixed Multimodal — best score config (boss-tp2-5r-450u-ramp90, 219,851,444):

```
python run_infertutor_experiment.py --label tp2-5r-450u-mixed-ramp90 --gpu-type H100 --replicas 5 --tp 2 --max-seqs 32 --max-batch-tokens 4096 --mode mixed --users 450 --duration 90 --ramp-up 90 --max-tokens 96 --no-fast-boot
python score_infertutor.py results_infertutor
modal app stop --yes infertutor-claude-tp2-5r-450u-mixed-ramp90
```

Boss Fight / Current Best Score — score-optimal density discovery (boss-10r-600u-ramp120):

```
python run_infertutor_experiment.py --label boss-10r-600u-ramp120 --gpu-type H100 --replicas 10 --max-seqs 32 --max-batch-tokens 4096 --mode text --users 600 --concurrent-inputs 128 --no-fast-boot --duration 90 --ramp-up 120 --max-tokens 96
python score_infertutor.py results_infertutor
modal app stop --yes infertutor-claude-boss-10r-600u-ramp120
```

Boss Fight / v2 Submission Reference (boss-8r-600u-ramp90 — score 364,851,394):

```
python run_infertutor_experiment.py --label boss-8r-600u-ramp90 --gpu-type H100 --replicas 8 --max-seqs 32 --max-batch-tokens 4096 --mode text --users 600 --concurrent-inputs 128 --no-fast-boot --duration 90 --ramp-up 90 --max-tokens 96
python score_infertutor.py results_infertutor
modal app stop --yes infertutor-claude-boss-8r-600u-ramp90
```

## 10. Summary & Conclusion

This capstone set out to deploy a real multimodal LLM serving system, measure it under concurrent load, and improve performance through principled inference engineering decisions. Over 70 experiments on H100 and B200 GPUs across two competition tracks, a boss fight, and a full extended study, the system evolved from a 50-

user eager baseline scoring 5,893,938 to a 10-GPU compiled configuration scoring 565,800,806 on H100 (a 96× improvement), with an observed all-time peak of 601,775,641 on B200. The current overall best score is 601,775,641, set by boss-b200-10r-600u-ramp100-r3 (B200 extended study, non-deterministic). The reliable H100 ceiling is 565,800,806, set by boss-10r-600u-ramp120 on May 30, 2026 — deterministic, 0.0% errors, reproducible. Experimentation continues through the June 14, 2026 submission deadline. Extended studies on Blackwell B200 hardware, FP8 KV cache quantization, tensor parallelism, GPU density ablations, and sequence-length confirmation are documented in Section 7.

## What Was Accomplished

Three distinct performance regimes were identified and optimized:

- **Single-GPU text throughput**: the dominant bottleneck was Python-side kernel dispatch overhead (~17 ms/token), not GPU compute. Compiled mode (CUDA graph) eliminated this entirely, dropping ITL from 17 ms to 6.8 ms and producing an 8.7× (773%) score increase on one GPU — the single largest gain of the entire project.
- **Multi-GPU scale-out**: horizontal replica scaling multiplied the compiled single-GPU result by 7.1×. The key condition for this to work was ensuring every replica completed CUDA graph compilation before peak load arrived, which required a 90-second ramp-up. Without it, replicas served in eager mode and the ITL advantage evaporated — confirmed by the anomalously low throughput in boss-8r-550u-ramp90.
- **Mixed multimodal track**: compiled mode was unavailable (variable image tensor shapes). The solution was to reduce per-step token budget (b2048) to force the chunked prefill scheduler to interleave decode steps more aggressively between image prefill chunks, and to distribute the prefill queue across 4 replicas. Together these brought TTFT from 2,785 ms down to 830 ms — below the 898 ms assignment reference.
- **Extended hardware, quantization, and density studies**: Blackwell B200 was tested against the identical boss fight configuration, confirming a 35% ITL reduction (6.0→3.9 ms) from HBM3e bandwidth — producing a 444M extended study score. Post-submission density ablations on H100 discovered that 60u/GPU is score-optimal at 10 replicas (vs. 75u/GPU at 8 replicas), pushing the H100 record to 565,800,806 — a 55% gain over the primary submission. Seven ablations across FP8 KV cache, MXFP8 KV, NVFP4, speculative decoding, TP=2 saturation, and B200 ramp-up non-determinism produced concrete findings on compatibility constraints and scale-dependent behavior that would not have been apparent from the primary experiments alone.

## The Central Insight

The most important lesson from this project is that diagnosis must precede optimization. Every experiment that added resources without identifying the actual bottleneck made the score worse. Scale-out at 200 users without compiled mode increased the GPU denominator without addressing the Python dispatch ceiling. Reducing pixel budget to save image compute increased scheduler fragmentation instead. Lowering the batch token budget to improve latency worsened GPU utilization for text traffic.

> The score formula rewards efficiency, not raw resource use. The primary submission at 75u/GPU reproduced the single-GPU knee at scale. The post-submission discovery refined this further: at 10 replicas, 60u/GPU is score-optimal because TTFT pressure from 750 concurrent connections becomes the binding constraint. Optimal density is not fixed — it is a function of replica count, network topology, and aggregate connection pressure. Measuring the TTFT-density curve at each scale point is the required engineering work before committing to a configuration.

## Key Findings Summarized

| Finding | Detail |
|---|---|
| **Best single technique** | Compiled mode (CUDA graph) — ITL −63%, score +773% (8.7×) on 1 GPU at no extra cost |
| **Overall best score** | 601,775,641: boss-b200-10r-600u-ramp100-r3 (B200 extended study, non-deterministic). TTFT 481 ms, ITL 3.6 ms, 1.7% errors. Observed peak across all 70+ experiments. |

**InferTutor Arena — Capstone Report**
Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14
**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**
**601,775,641**
B200 · r3 · 10r · 600u (observed peak)
H100 reliable: 565,800,806

| Finding | Detail |
|---|---|
| **Reliable H100 ceiling** | 565,800,806: boss-10r-600u-ramp120 (10 H100 compiled replicas, 600u, 120s ramp). Deterministic: TTFT 308 ms, ITL 5.4 ms, 0.0% errors. 96× over baseline. |
| **v2 submission reference** | 8 H100 compiled replicas, 600 users, ci128, 90 s ramp-up — score 364,851,394 (75u/GPU, 0.0% errors). 62× over baseline. |
| **60u/GPU vs 75u/GPU insight** | Score-optimal density is replica-count-dependent: 75u/GPU maximizes throughput per GPU; at 10 replicas, 60u/GPU drops TTFT from 1,074 ms to 308 ms (3.5×), producing 3.4× score gain despite 20% fewer users |
| **TP=2 saturation cliff** | TP=2 (5 replicas × 2 GPUs) works at 75u/replica (score 208M); at 90u/replica, TTFT spikes to 4,879 ms and score collapses to 9M. The operating window is narrow. |
| **B200 ramp100 non-determinism** | 6 standard ramp100 runs: 601M, 587M, 573M, 499M, 486M (compiled), 354M (partial) — 83% compile rate. ramp110 always compiles (489M); longer ramps (115–130s) compile but score 431M–480M. The 601M overall best cannot be reliably reproduced; H100-10r 565M is the deterministic ceiling. |
| **seq32 confirmed optimal** | seq48 on H100 raised TTFT 308→377 ms, score fell 24.9% (565M→424M). seq24 on B200 raised ITL 3.6→4.1 ms, score fell 13.5% (601M→520M). seq32 is optimal on both architectures. |
| **Most surprising failure** | Halving mm-max-pixels made TTFT worse (+138 %) due to scheduler fragmentation from irregular image patch shapes |
| **Batch token rule** | b2048 helps mixed traffic (chunks large image prefills); b2048 hurts text traffic (reduces GPU utilization) |
| **Scaling rule** | Horizontal replicas only multiply existing per-GPU efficiency — they do not fix a per-step bottleneck |
| **Track 1 result** | 4 replicas + b2048 achieved 830 ms TTFT, beating the 898 ms assignment reference with zero errors. TP=2 mixed (5r×2GPU, 450u) later achieved 219,851,444 — the new Track 1 ceiling (Section 7.12) |
| **Ramp-up criticality** | 90 s ramp-up was the decisive parameter — 75s still produced cold-start errors; 90s gave all 8 replicas time to compile CUDA graphs before peak load, yielding 0.0% errors |
| **Prefix caching** | Disabling it raised TTFT 133% at 125 users, confirming shared system prompt reuse is a meaningful saving |
| **B200 hardware advantage** | Blackwell B200 reduced ITL from 6.0 ms to 3.9 ms (−35%) on the identical boss fight configuration — driven by HBM3e bandwidth (8 TB/s vs H100's 3.35 TB/s). Observed B200 best: 601,775,641 |
| **FP8 KV on H100 — hurts** | FP8 KV cache raised ITL from 6.0 ms to 8.2 ms (+37%) on H100. Hopper's dequantization step on every decode access costs more than the memory savings. BF16 KV cache is optimal on H100 |
| **MXFP8 KV on B200 — concurrency-dependent** | MXFP8 KV helps at low concurrency on B200 (1 GPU: ITL 3.3→3.2 ms, +9% score) but hurts at high concurrency (8 GPUs: ITL 3.9→4.7 ms, −5% score). BF16 KV remains optimal at 75 users/replica |

## Conclusion

Inference engineering is ultimately about understanding where time is actually being spent. The GPU is rarely the bottleneck at model sizes below 10B parameters and moderate concurrency — the bottleneck is more often the serving framework's control plane, the scheduler's batching decisions, or the memory subsystem's ability to keep KV state resident. This project demonstrated that a 4B model on a single H100 can serve 125 text users at 6.8 ms ITL with zero errors when the Python dispatch overhead is removed, and that this efficiency scales to 10 GPUs when each replica is given time to warm up and the per-GPU user density is tuned for the aggregate TTFT pressure — not just for individual-GPU throughput.

The extended studies in Section 7 pushed beyond the primary H100 results. B200 hardware reduced ITL from 6.0 ms to 3.9 ms, producing a 444M extended study score on 8 replicas. Post-submission density ablations discovered that 60u/GPU is score-optimal at 10 H100 replicas, producing a reliable H100 record of 565,800,806 — a 55% improvement over the primary submission at only 25% more GPUs. B200 at 10 replicas produced an observed all-time best of 601,775,641 (ramp100-r3, TTFT 481 ms, ITL 3.6 ms) across six standard ramp100 runs at an 83% compile rate — scores ranged from 354M (partial compile) to 601M (compiled); mean when compiled 549M. ramp110 proved the deterministic alternative at 489M. Two seq-length ablations closed the sequence search space: seq48 worsened the H100 score by 24.9% and seq24 worsened the B200 score by 13.5%, confirming seq32 as optimal on both architectures. The TP=2 saturation cliff at 90u/replica collapsed score by

22×. FP8 and MXFP8 KV cache ablations confirmed that the optimal quantization strategy is hardware-generation-dependent: BF16 KV is best on H100, BF16 KV also best on B200 at high concurrency, and MXFP8 KV provides marginal gains at low concurrency only.

Experimentation continues through the June 14, 2026 deadline. The techniques that remain genuinely unexplored — TP=4 on 2 replicas × 4 GPUs, a finer density sweep around 55–65u/GPU at 10 replicas, n-gram speculative decoding in eager mode, and request-type-aware routing for mixed traffic — each target a specific remaining layer of the bottleneck stack. Given the 96× improvement achieved from baseline to the current best across all experiments, the compounding potential of even two of those remaining techniques suggests the score can be pushed higher. The foundation built here — a reliable measurement pipeline, a clear scoring model, a precise understanding of each knob's scale-dependent effect, and the key insight that TTFT pressure from aggregate connections is the binding constraint at 10 replicas — is the prerequisite for that next phase.

## References

**[1]** R. Dandekar, R. Dandekar, and S. Panat, "Inference Engineering: The Definitive Workshop Guide," Vizuara, 2026. — Primary course text and instructional framework for the Vizuara Inference Engineering Workshop. The InferTutor Arena capstone, scoring formula, workload definitions, and experimental methodology in this report were designed and administered by the authors of this guide.

**[2]** O. Awoga, "Production LLM Inference Engineering — Close to the Metal: LLM Inference from First Principles," 52 Chapters · 29 Appendices · May 2026. Available at: https://maximumbeings.github.io/close-to-the-metal/index.html — A practitioner's deep guide covering GPU memory hierarchies, attention arithmetic, transformer internals, production scheduling, multi-GPU serving, quantization, and cost engineering. Core concepts are grounded in vLLM and llama.cpp; additional chapters address SGLang, TensorRT-LLM, and Blackwell hardware. Provided conceptual grounding for the inference engineering decisions throughout this capstone.

**[3]** P. Kiely, "Inference Engineering," Baseten Books, Baseten Labs, Inc., 2026. — A practitioner-oriented reference on LLM inference systems engineering covering deployment, optimization, and production serving patterns. Informed the serving design and optimization strategy applied in this project.

**[4]** C. Wang and P. Hu, Hands-On LLM Serving and Optimization: Hosting LLMs at Scale, O'Reilly Media, Inc., May 2026. ISBN 9798341621480, 374 pages. — A hands-on practitioner guide to deploying and optimizing large language models at scale, covering serving architecture, batching strategies, quantization, and production-grade performance tuning. Informed the serving design trade-offs and hardware-generation comparison methodology applied in this capstone.

**[5]** W. Kwon, Z. Li, S. Zhuang, Y. Sheng, G. Zheng, H. Zhang, J. E. Gonzalez, I. Stoica, and E. P. Xing, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. ACM Symposium on Operating Systems Principles (SOSP), 2023. — Foundation of vLLM's KV cache management; the PagedAttention paper underpinning the serving engine used throughout this project.

**[6]** G. Yu, J.-W. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2022. — Introduced iteration-level continuous batching; the scheduling strategy adopted by vLLM that allows new requests to enter the batch between decode steps, directly influencing the TTFT vs throughput trade-off studied here.

**[7]** T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in Proc. International Conference on Learning Representations (ICLR), 2024. — The attention kernel used by vLLM on H100 GPUs. IO-aware tiling reduces HBM round-trips during the attention step, contributing to the low per-step compute observed in compiled mode.

**[8]** Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in Proc. International Conference on Machine Learning (ICML), 2023. — Theoretical basis for the speculative decoding technique discussed in Sections 6.7 and 7.6. Demonstrates that a small draft model can generate K candidate tokens per step for the large model to verify in a single parallel forward pass, multiplying effective decode throughput by 2–4× at high acceptance rates.

**[9]** Qwen Team, Alibaba Cloud, "Qwen2.5-VL Technical Report," arXiv:2502.13923, 2025. — Technical documentation for the Qwen VL model family used in this project (Qwen3-VL-4B-Instruct). Covers the vision encoder architecture, dynamic-resolution tiling, and grouped-query attention design that influenced KV cache sizing and the mm-max-pixels behavior observed in experiments.

**[10]** NVIDIA Corporation, "CUDA Graphs," CUDA C++ Programming Guide, v12.x, 2024. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs — Reference documentation for the CUDA graph capture and replay

mechanism that vLLM's compiled (--no-fast-boot) mode relies on. The elimination of per-kernel launch overhead documented here explains the 65% ITL reduction (19.6 ms → 6.8 ms) measured in EXP-13.

## Appendix A. Full Raw Leaderboard — All 70 Runs

Complete output of score_infertutor.py across all result JSON files, sorted by score descending. Runs marked † are B200 extended-study runs (not counted in the primary H100 experiment matrix). Post-submission H100 runs and B200 10r extended study runs are included. Runs with score = 0 or throughput < 10 c/s are partial cold-start exits — the endpoint was still warming up when the load test began; these are retained for transparency but excluded from analysis. Gold background = primary submission runs and new extended study record.

| Run Label | Mode | Users | GPUs | Err % | TTFT p95 | ITL p95 | Thruput | Score | Configuration |
|---|---|---|---|---|---|---|---|---|---|
| boss-b200-10r-600u-ramp100 (r3) † | text | 600 | 10 | 1.7% | 481 ms | 3.6 ms | 17,498 | 601,775,641 | B200 · 10r×1GPU · compiled · b4096 · seq32 · BF16 KV · 100s ramp · OVERALL BEST |
| boss-b200-10r-600u-ramp100 (r1) † | text | 600 | 10 | 0.8% | 536 ms | 3.3 ms | 17,455 | 587,515,805 | B200 · 10r×1GPU · compiled · b4096 · seq32 · BF16 KV · 100s ramp (non-deterministic) |
| **boss-10r-600u-ramp120** | text | 600 | 10 | 0.0% | 308 ms | 5.4 ms | 15,601 | **565,800,806** | H100 · 10r×1GPU · compiled · b4096 · seq32 · BF16 KV · 120s ramp · 60u/GPU · H100 reliable ceiling |
| boss-10r-650u-ramp120 | text | 650 | 10 | 0.0% | 367 ms | 5.6 ms | 16,449 | 523,925,209 | H100 · 10r×1GPU · compiled · b4096 · seq32 · BF16 KV · 120s ramp · 65u/GPU |
| boss-b200-10r-600u-ramp100-seq24 † | text | 600 | 10 | 1.5% | 489 ms | 4.1 ms | 17,583 | 520,691,386 | B200 · 10r×1GPU · compiled · b4096 · seq24 · BF16 KV · 100s ramp (seq24 ablation — seq32 superior) |
| boss-b200-10r-600u-ramp100 (r4) † | text | 600 | 10 | 1.2% | 527 ms | 3.9 ms | 17,462 | 499,230,397 | B200 · 10r×1GPU · compiled (marginal) · b4096 · seq32 · BF16 KV · 100s ramp |
| boss-b200-10r-600u-ramp110 † | text | 600 | 10 | 0.0% | 601 ms | 3.5 ms | 17,142 | 489,330,868 | B200 · 10r×1GPU · compiled · b4096 · seq32 · |

## InferTutor Arena — Capstone Report

Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14

**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**

**601,775,641**

B200 · r3 · 10r · 600u (observed peak)

H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | Err % | TTFT p95 | ITL p95 | Thruput | Score | Configuration |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | BF16 KV · 110s ramp · deterministic |
| boss-b200-10r-600u-ramp100 (r5) † | text | 600 | 10 | 1.9 % | 554 ms | 3.3 ms | 17,580 | 573,727,942 | B200 · 10r×1GPU · compiled · b4096 · seq32 · BF16 KV · 100s ramp (non-deterministic) |
| boss-b200-10r-600u-ramp100 (r6) † | text | 600 | 10 | 1.6 % | 559 ms | 3.8 ms | 17,522 | 486,061,520 | B200 · 10r×1GPU · compiled · b4096 · seq32 · BF16 KV · 100s ramp (non-deterministic) |
| boss-8r-600u-ramp90-B200 † | text | 600 | 8 | 1.7 % | 744 ms | 3.9 ms | 17,589 | 444,362,351 | B200 · 8r×1GPU · compiled · b4096 · seq32 · BF16 KV · 90s ramp |
| boss-10r-600u-ramp120-seq48 | text | 600 | 10 | 0.0 % | 377 ms | 5.7 ms | 15,145 | 424,934,532 | H100 · 10r×1GPU · compiled · b4096 · seq48 · BF16 KV · 120s ramp (seq48 ablation — seq32 superior) |
| b200-8r-600u-ramp90-mxfp8kv † | text | 600 | 8 | 1.4 % | 664 ms | 4.7 ms | 17,854 | 420,047,828 | B200 · 8r×1GPU · compiled · b4096 · seq32 · MXFP8 KV · 90s ramp |
| **boss-8r-600u-ramp90** | text | 600 | 8 | 0.0 % | 583 ms | 6.0 ms | 17,100 | **364,851,394** | H100 · 8r×1GPU · compiled · b4096 · seq32 · BF16 KV · 90s ramp |
| boss-b200-10r-600u-ramp100 (r2) † | text | 600 | 10 | 0.8 % | 581 ms | 5.0 ms | 17,364 | 354,755,892 | B200 · 10r×1GPU · partial compile · b4096 · seq32 · BF16 KV · 100s ramp (non-deterministic) |
| boss-8r-650u-ramp90 | text | 650 | 8 | 1.7 % | 655 ms | 6.2 ms | 17,821 | 350,833,555 | H100 · 8r×1GPU · compiled · b4096 · seq32 · BF16 KV · 90s ramp |
| boss-8r-550u-ci128 | text | 550 | 8 | 1.1 % | 635 ms | 6.3 ms | 17,425 | 297,337,012 | H100 · 8r×1GPU · compiled · b4096 · seq32 · ci128 |
| boss-8r-600u-ramp90-kvcfp8-seq48 | text | 600 | 8 | 0.0 % | 672 ms | 6.6 ms | 17,442 | 295,264,172 | H100 · 8r×1GPU · compiled · b4096 · seq48 · FP8 KV · 90s ramp |
| boss-10r-500u-ramp120 | text | 500 | 10 | 0.0 % | 435 ms | 5.5 ms | 12,914 | 272,183,455 | H100 · 10r×1GPU · compiled · |

## InferTutor Arena — Capstone Report

Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14

**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**

**601,775,641**

B200 · r3 · 10r · 600u (observed peak)

H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | Err % | TTFT p95 | ITL p95 | Thruput | Score | Configuration |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | b4096 · seq32 · 120s ramp · 50u/GPU (below optimal density) |
| boss-8r-600u-ci128 | text | 600 | 8 | 5.0 % | 876 ms | 5.9 ms | 18,916 | 259,405,996 | H100 · 8r×1GPU · compiled · b4096 · seq32 · ci128 (queue saturation) |
| boss-10r-550u-ramp120 | text | 550 | 10 | 0.0 % | 510 ms | 5.9 ms | 13,597 | 247,392,980 | H100 · 10r×1GPU · compiled · b4096 · seq32 · 120s ramp · 55u/GPU (late compile trigger) |
| boss-8r-550u-ramp90 | text | 550 | 8 | 0.1 % | 537 ms | 5.9 ms | 10,969 | 239,478,093 | H100 · 8r×1GPU · compiled · b4096 · seq32 · 90s ramp (cold-start issue) |
| boss-8r-650u-ramp90-kvcfp8 | text | 650 | 8 | 0.0 % | 721 ms | 8.7 ms | 17,770 | 229,390,360 | H100 · 8r×1GPU · compiled · b4096 · seq32 · FP8 KV · 90s ramp |
| boss-tp2-5r-450u-ramp90 (mixed) | mixed | 450 | 10 | 0.0 % | 632 ms | 4.8 ms | 14,792 | 219,851,444 | H100 · 5r×2GPU · TP=2 · compiled · b4096 · seq32 · mixed mode · NEW TRACK 1 BEST |
| tp2-5r-500u-mixed-ramp90 | mixed | 500 | 10 | 0.0 % | 763 ms | 6.3 ms | 15,176 | 158,962,950 | H100 · 5r×2GPU · TP=2 · b4096 · seq32 · mixed · 500u knee exceeded |
| tp2-5r-550u-mixed-ramp90 | mixed | 550 | 10 | 0.0 % | 958 ms | 5.0 ms | 15,722 | 178,916,605 | H100 · 5r×2GPU · TP=2 · b4096 · seq32 · mixed · 550u oversaturated |
| tp2-5r-450u-mixed-b2048 | mixed | 450 | 10 | 0.0 % | 3,223 ms | 33.0 ms | 4,314 | 1,825,299 | H100 · 5r×2GPU · TP=2 · b2048 · seq32 · mixed · b2048 incompatible with TP=2 |
| boss-8r-600u-ramp90-kvcfp8 | text | 600 | 8 | 0.0 % | 702 ms | 8.2 ms | 16,818 | 219,447,305 | H100 · 8r×1GPU · compiled · b4096 · seq32 · FP8 KV · 90s ramp |
| boss-tp2-5r-375u-ramp90 | text | 375 | 10 | 0.0 % | 547 ms | 5.0 ms | 15,294 | 208,596,784 | H100 · 5r×2GPU · TP=2 · compiled · b4096 · seq32 · NVLink all-reduce |
| boss-b200-10r-600u-ramp90 † | text | 60 | 10 | 1.0 | 710 | 7.1 | 17,736 | 207,794,5 | B200 · 10r×1GPU |

## InferTutor Arena — Capstone Report

Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14

**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**

**601,775,641**

B200 · r3 · 10r · 600u (observed peak)

H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | Err % | TTFT p95 | ITL p95 | Thruput | Score | Configuration |
|---|---|---|---|---|---|---|---|---|---|
| | | 0 | | % | ms | ms | | 88 | · eager (not compiled) · b4096 · seq32 · 90s ramp insufficient |
| boss-8r-550u-ml4096-seq64 | text | 550 | 8 | 0.1 % | 666 ms | 7.6 ms | 14,752 | 200,536,427 | H100 · 8r×1GPU · compiled · b4096 · seq64 · ml4096 |
| boss-8r-600u-ramp120-B200 † | text | 600 | 8 | 0.0 % | 989 ms | 5.3 ms | 12,983 | 186,047,228 | B200 · 8r×1GPU · compiled · b4096 · seq32 · 120s ramp |
| boss-b200-10r-550u-ramp100 † | text | 550 | 10 | 0.0 % | 667 ms | 7.2 ms | 16,081 | 183,225,927 | B200 · 10r×1GPU · not compiled (ITL 7.2ms) · b4096 · seq32 · 100s ramp · 55u/GPU |
| boss-8r-500u-ci128 | text | 500 | 8 | 0.5 % | 692 ms | 8.8 ms | 17,436 | 177,636,936 | H100 · 8r×1GPU · compiled · b4096 · seq32 · ci128 |
| boss-10r-750u-ramp120 | text | 750 | 10 | 0.0 % | 1,074 ms | 6.4 ms | 15,219 | 165,115,545 | H100 · 10r×1GPU · compiled · b4096 · seq32 · 120s ramp · 75u/GPU (TTFT-density failure) |
| b200-1r-75u-mxfp8kv † | text | 75 | 1 | 0.0 % | 519 ms | 3.2 ms | 2,675 | 119,563,095 | B200 · 1r×1GPU · compiled · b4096 · seq32 · MXFP8 KV |
| b200-1r-75u-bf16 † | text | 75 | 1 | 0.0 % | 552 ms | 3.3 ms | 2,649 | 109,654,482 | B200 · 1r×1GPU · compiled · b4096 · seq32 · BF16 KV |
| boss-b200-10r-600u-ramp103 † | text | 600 | 10 | 0.0 % | 976 ms | 6.8 ms | 16,138 | 146,913,895 | B200 · 10r×1GPU · not compiled (ITL 6.8ms) · b4096 · seq32 · 103s ramp · failed this run |
| boss-8r-800u-ramp90-B200 † | text | 800 | 8 | 0.0 % | 3,472 ms | 4.7 ms | 15,554 | 95,026,403 | B200 · 8r×1GPU · compiled · b4096 · seq32 · 90s ramp (oversaturated) |
| boss-text-8r-600u | text | 600 | 8 | 5.5 % | 1,511 ms | 11.4 ms | 19,660 | 80,646,313 | H100 · 8r×1GPU · compiled · b4096 · seq32 · ci64 (pre-ramp config) |
| boss-tp4-2r-300u-ramp120 | text | 300 | 8 | 0.0 % | 941 ms | 4.1 ms | 7,157 | 69,577,107 | H100 · 2r×4GPU · TP=4 · compiled · b4096 · seq32 · 120s ramp · |

**InferTutor Arena — Capstone Report**
Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14
**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**
**601,775,641**
B200 · r3 · 10r · 600u (observed peak)
H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | Err % | TTFT p95 | ITL p95 | Thruput | Score | Configuration |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | 150u/replica |
| boss-text-8r-800u | text | 800 | 8 | 13.1% | 3,694 ms | 6.7 ms | 18,234 | 64,159,316 | H100 · 8r×1GPU · compiled · b4096 · seq32 · ci64 |
| boss-text-8r-1000u | text | 1000 | 8 | 20.4% | 5,276 ms | 7.1 ms | 20,324 | 53,917,925 | H100 · 8r×1GPU · compiled · b4096 · seq32 · ci64 |
| **compiled-1r-125u** | text | 125 | 1 | 0.0 % | 1,542 ms | 6.8 ms | 4,285 | **51,455,660** | H100 · 1r×1GPU · compiled · b4096 · seq32 · BF16 KV |
| boss-10r-750u-ramp90 | text | 750 | 10 | 0.0 % | 1,860 ms | 12.8 ms | 15,038 | 47,276,760 | H100 · 10r×1GPU · compiled (partial) · b4096 · seq32 · 90s ramp (insufficient — replicas served eager) |
| boss-tp2-5r-300u-ramp120 | text | 300 | 10 | 0.0 % | 1,204 ms | 5.0 ms | 7,055 | 35,479,487 | H100 · 5r×2GPU · TP=2 · compiled · b4096 · seq32 · 120s ramp · 60u/replica |
| boss-10r-625u-ramp120 | text | 625 | 10 | 0.0 % | 2,196 ms | 7.7 ms | 8,881 | 32,669,427 | H100 · 10r×1GPU · compiled · b4096 · seq32 · 120s ramp · 62.5u/GPU (TTFT collapse — over 60u/GPU optimal) |
| boss-tp2-5r-450u-text-ramp90 | text | 450 | 10 | 0.0 % | 4,879 ms | 5.8 ms | 5,840 | 9,366,571 | H100 · 5r×2GPU · TP=2 · compiled · b4096 · seq32 · 90u/replica — saturation cliff |
| knee-1r-75u | text | 75 | 1 | 0.0 % | 1,281 ms | 16.2 ms | 1,964 | 7,111,380 | H100 · 1r×1GPU · eager · b4096 · seq32 |
| baseline-text | text | 50 | 1 | 0.0 % | 608 ms | 19.6 ms | 1,406 | 5,893,938 | H100 · 1r×1GPU · eager · b4096 · seq32 (starter config) |
| knee-1r-100u | text | 100 | 1 | 0.0 % | 2,416 ms | 16.1 ms | 1,937 | 4,984,026 | H100 · 1r×1GPU · eager · b4096 · seq32 |
| batch-small-1r-125u | text | 125 | 1 | 0.0 % | 3,485 ms | 15.5 ms | 2,049 | 4,736,629 | H100 · 1r×1GPU · eager · b2048 · seq32 |
| prefix-off-1r-125u | text | 125 | 1 | 0.0 % | 3,591 ms | 16.2 ms | 1,966 | 4,218,377 | H100 · 1r×1GPU · eager · b4096 · seq32 · prefix cache OFF |

## InferTutor Arena — Capstone Report

Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14

**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**

**601,775,641**

B200 · r3 · 10r · 600u (observed peak)

H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | Err % | TTFT p95 | ITL p95 | Thruput | Score | Configuration |
|---|---|---|---|---|---|---|---|---|---|
| knee-1r-125u | text | 125 | 1 | 0.1 % | 3,512 ms | 16.2 ms | 1,869 | 4,105,702 | H100 · 1r×1GPU · eager · b4096 · seq32 |
| baseline-text-200u | text | 200 | 1 | 0.0 % | 6,891 ms | 17.8 ms | 2,014 | 3,284,129 | H100 · 1r×1GPU · eager · b4096 · seq32 |
| **mixed-r4-b2048** | mixed | 120 | 4 | 0.0 % | 830 ms | 31.8 ms | 2,697 | **3,061,971** | H100 · 4r×1GPU · eager · b2048 · seq32 · mixed workload |
| scale-2r-100u | text | 100 | 2 | 0.0 % | 1,787 ms | 23.2 ms | 2,463 | 2,976,312 | H100 · 2r×1GPU · eager · b4096 · seq32 |
| batch-wide-1r-125u | text | 125 | 1 | 0.0 % | 4,501 ms | 22.2 ms | 1,604 | 2,011,089 | H100 · 1r×1GPU · eager · b8192 · seq32 |
| mixed-r4-lowpx | mixed | 120 | 4 | 0.0 % | 1,977 ms | 22.4 ms | 2,757 | 1,865,134 | H100 · 4r×1GPU · eager · b4096 · seq32 · low pixel budget · mixed |
| scale-out-2r | text | 200 | 2 | 0.0 % | 7,245 ms | 21.2 ms | 2,599 | 1,692,831 | H100 · 2r×1GPU · eager · b4096 · seq32 |
| mixed-r2-batch2048 | mixed | 100 | 2 | 0.1 % | 2,144 ms | 29.7 ms | 1,983 | 1,556,922 | H100 · 2r×1GPU · eager · b2048 · seq32 · mixed workload |
| final-mixed | mixed | 120 | 4 | 0.0 % | 2,074 ms | 30.4 ms | 2,789 | 1,325,393 | H100 · 4r×1GPU · eager · b4096 · seq32 · mixed workload |
| mixed-r2-baseline | mixed | 100 | 2 | 0.0 % | 2,785 ms | 29.8 ms | 1,865 | 1,123,216 | H100 · 2r×1GPU · eager · b4096 · seq32 · mixed workload |
| mixed-r4-150u-b2048 | mixed | 150 | 4 | 0.0 % | 3,114 ms | 32.0 ms | 2,795 | 1,050,285 | H100 · 4r×1GPU · eager · b2048 · seq32 · mixed workload |
| knee-1r-125u (partial) | text | 125 | 1 | 0.0 % | 735 ms | 14.2 ms | 16 | 190,025 | H100 · 1r×1GPU · eager · b4096 · seq32 (partial — warming) |
| knee-1r-100u (partial) | text | 100 | 1 | 0.0 % | 621 ms | 14.5 ms | 16 | 177,283 | H100 · 1r×1GPU · eager · b4096 · seq32 (partial — warming) |
| compiled-1r-125u (partial) | text | 125 | 1 | 0.0 % | 383 ms | 4.5 ms | 1 | 76,937 | H100 · 1r×1GPU · compiled · b4096 · seq32 (partial — cold start) |

**InferTutor Arena — Capstone Report**
Vizuara Inference Engineering Workshop | 2026-05-30 | Deadline: 2026-06-14
**OLUWASEYI AWOGA [OAWOGA@SEAS.UPENN.EDU]**

**CURRENT BEST SCORE**
**601,775,641**
B200 · r3 · 10r · 600u (observed peak)
H100 reliable: 565,800,806

| Run Label | Mode | Users | GPUs | Err % | TTFT p95 | ITL p95 | Thruput | Score | Configuration |
|---|---|---|---|---|---|---|---|---|---|
| smoke | text | 5 | 1 | 0.0 % | 445 ms | 19.5 ms | 129 | 74,308 | H100 · 1r×1GPU · eager · b4096 · seq32 (smoke test, 5 users) |
| knee-1r-75u (partial) | text | 75 | 1 | 0.0 % | 514 ms | 13.3 ms | 5 | 52,279 | H100 · 1r×1GPU · eager · b4096 · seq32 (partial — warming) |
| knee-1r-100u (cold exit) | text | 100 | 1 | 0.0 % | — | — | 0 | 0 | H100 · 1r×1GPU · eager — cold exit, no steady-state data |
| knee-1r-75u (cold exit) | text | 75 | 1 | 0.0 % | — | — | 0 | 0 | H100 · 1r×1GPU · eager — cold exit, no steady-state data |

† B200 extended-study runs (Blackwell hardware, Section 7). Not counted in the primary H100 experiment matrix. Post-submission H100 runs (boss-10r-*, boss-tp2-5r-450u-*) were executed after the v2 submission was packaged; results are documented in Section 7.11. Partial / cold-exit runs (score ≤ 190,025) are retained for transparency; the endpoint was still warming up during these load tests and did not reach steady state.